UNIVERSIDADE LaSalle
Viver é evoluir.

NOVA GRADUAÇÃO UNIVERSIDADE LA SALLE

Algoritmos e PROGRAMAÇÃO



INOVAÇÃO E TECNOLOGIA

NOVA GRADUAÇÃO
UNIVERSIDADE LA SALLE

Algoritmos e PROGRAMAÇÃO

Aline Duarte Riva
Mozart Lemos De Siqueira
(Orgs.)

Editora LaSalle

UNIVERSIDADE LA SALLE

Reitor
Prof. Dr. Cledes Casagrande - Fsc

Vice-Reitor
Prof. Me. Ir. Eucledes Fábio Casagrande

Pró-Reitor de Administração
Vitor Benites

PRODUÇÃO DE CONTEÚDO

Coordenador de Produção
Profª Me. Michele de Mattos Kreme

Equipe de Produção de Conteúdo
Bruno Giordani Faccio
Daiane dos Santos Costa
Gabriela Rossa
Ingrid Rais da Silva
Jorge Fabiano Mendez
Luiz Eduardo Campesato Rocha
Vinicius Silveira Martins

Projeto Gráfico, Editoração, Revisão e Produção
Equipe de Produção de Conteúdo Universidade La Salle - Canoas, RS

1ª Edição

Atualizada em:
Janeiro de 2025

© 2025 por Universidade La Salle

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer meio, eletrônico ou mecânico (fotocópia, gravação), ou qualquer tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização por escrito da Universidade La Salle.

Editora LaSalle

Universidade La Salle Canoas | Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500 | eadproducao@unilasalle.edu.br

APRESENTAÇÃO

Prezado estudante,

A equipe da EaD La Salle sente-se honrada em entregar a você este material didático. Ele foi produzido com muito cuidado para que cada Unidade de estudos possa contribuir com seu aprendizado da maneira mais adequada possível à modalidade que você escolheu para estudar: a modalidade a distância. Temos certeza de que o conteúdo apresentado será uma excelente base para o seu conhecimento e para sua formação. Por isso, indicamos que, conforme as orientações de seus professores e tutores, você reserve tempo semanalmente para realizar a leitura detalhada dos textos deste livro, buscando sempre realizar as atividades com esmero a fim de alcançar o melhor resultado possível em seus estudos. Destacamos também a importância de questionar, de participar de todas as atividades propostas no ambiente virtual e de buscar, para além de todo o conteúdo aqui disponibilizado, o conhecimento relacionado a esta disciplina que está disponível por meio de outras bibliografias e por meio da navegação online.

Desejamos a você um excelente módulo e um produtivo ano letivo. Bons estudos!

Nome da Disciplina

APRESENTANDO OS ORGANIZADORES

Aline Duarte Riva

Mestre em Ciência da Computação pela Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS) - Conclusão em março/2011. Possui graduação em Ciência da Computação pela ULBRA (2002) e MBA em Gestão Estratégica da Tecnologia da Informação pela FGV/RS (2004). Atualmente trabalha como professora no Unilasalle e como membro do escritório de projetos.

Mozart Lemos De Siqueira

Doutor em Ciência da Computação desde 2009 e mestre desde 2002 ambos pelo PPGC da UFRGS. Graduado como Bacharel em Informática em 1998 pela Universidade Luterana do Brasil, de Canoas. Inicio sua carreira como professor no ensino superior em 2002, na Universidade La Salle, no Curso de Bacharelado em Ciência da Computação. Tem interesse em temas como Processamento Paralelo, Arquitetura de Computadores, Sistemas Operacionais e Introdução a Programação. Atualmente atua como professor em disciplinas das áreas de interesse e é coordenador do Curso de Ciência da Computação e do Curso Superior de Tecnologia em Analise e Desenvolvimento de Sistemas, este último na modalidade de ensino à distância. Participa de comitês internos na universidade, como o comitê de TI, o comitê de ética e pesquisa, além de fazer parte do Núcleo Docente Estruturante dos cursos que coordena.

Apresentação da Disciplina

Olá! Seja bem-vindo(a) à disciplina de Algoritmos e Programação. Através dela, os alunos desenvolvem habilidades essenciais para a resolução de problemas computacionais, compreendendo desde os conceitos básicos de algoritmos até a implementação de soluções em uma linguagem de programação moderna.

Este e-book foi estruturado para fornecer uma base sólida e progressiva, dividindo o aprendizado em quatro unidades principais. Inicialmente, apresentamos os conceitos fundamentais, como o que é um algoritmo, sua representação e a importância da programação estruturada. Também exploramos os Ambientes Integrados de Desenvolvimento (IDEs) e introduzimos a linguagem Python, destacando sua versatilidade e facilidade de uso.

Na segunda unidade, aprofundamos as estruturas básicas de controle, com ênfase na sequência de comandos e na manipulação de variáveis. Apresentamos operadores aritméticos, relacionais e lógicos, além das funções de entrada e saída de dados, essenciais para a interatividade dos programas.

A terceira unidade aborda as estruturas de seleção e repetição, fundamentais para a construção de algoritmos dinâmicos e eficientes. Exploramos os comandos condicionais if, elif e else, bem como os laços de repetição while e for. Também introduzimos vetores e matrizes, abordando a utilização da biblioteca NumPy para manipulação de arrays.

Por fim, a quarta unidade tratará da programação modular, destacando a importância da organização do código em módulos reutilizáveis, tornando o desenvolvimento mais estruturado e eficiente.

Além da teoria, cada unidade conta com exercícios de fixação, incentivando a prática e a experimentação, pois acreditamos que a melhor maneira de aprender programação é a prática e a repetição.

Bons estudos!

Sumário

UNIDADE 1

Introdução a Algoritmos e Programação....9
Objetivo Geral 9
Objetivos Específicos 9
Questões Contextuais....9
1.1 Conceitos Básicos de Algoritmos....10
1.1.1 Algoritmos....11
1.2 O Paradigma da Programação Estruturada....17
1.3 o Ambiente Integrado de Desenvolvimento 20
1.3.1 O IDE Pycharm 22
1.3.2 O IDE Vscode.dev 25
1.4 A Linguagem de Programação Python 27
1.4.1 Oi Mundo Python....32
Síntese da Unidade....34
Bibliografia....35

UNIDADE 2

Estruturas Básicas de Controle - Sequência....37
Objetivo Geral 37
Objetivos Específicos 37
Questões Contextuais....37
2.1 Executando Programas em Python....38
2.2 Introdução ao Tratamento de Variáveis....42
2.3 Operadores Aritméticos 49
2.4 Operadores Relacionais e Lógicos....51
2.5 Operações de Entrada e Saída de Dados....55
Síntese da Unidade....58
Bibliografia....59

UNIDADE 3

Estruturas de Controle - Seleção e Repetição **61**
Objetivo Geral **61**
Objetivos Específicos **61**
Questões Contextuais **61**
3.1 IF, ELIF E ELSE **62**
3.1.1 Considerações Sobre Operadores Lógicos em Condições 67
3.1.2 Curiosidades Sobre a Condicional if 68
3.2 Laço While **69**
3.3 Laço for **76**
3.4 Vetores e Matrizes **81**
Síntese da Unidade **86**
Bibliografia **87**

UNIDADE 4

Programação Modular **89**
Objetivo Geral **89**
Objetivos Específicos **89**
Questões Contextuais **89**
4.1 Questões Fundamentais Sobre Programação Modular **90**
4.2 Formas de uma Função **96**
4.3 Tuplas, Listas e Dicionários **103**
4.4 Introdução a Manipulação de Arquivos **108**
4.5 Projetos com Mais de um Arquivo Fonte **113**
Síntese da Unidade **117**
Bibliografia **118**

Introdução a Algoritmos e Programação

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Apresentar um panorama geral da disciplina, introduzindo termos, conceitos e ferramentas necessárias

Objetivos Específicos

- Introdução à disciplina, apresentação de termos e conceitos.
- Apresentar a base necessária para iniciar o desenvolvimento.
- Apresentar a linguagem de programação Python

Questões Contextuais

- Por que aprender a programar?
- Qual a diferença entre o algoritmo e o programa?
- Por que a linguagem de programação é necessária?

Editora LaSalle

1.1 Conceitos Básicos de Algoritmos

Uma carreira na área de desenvolvimento de software é construída durante muito tempo de aprendizado. Nem todas as nuances e detalhes necessários serão aprendidos durante a carreira acadêmica, pois a jornada é longa. Este material foi organizado para que obtenha as ferramentas necessárias para dar início a sua carreira, para aprender uma linguagem de programação e depois outras, mais frameworks e arquiteturas que surgirão com o passar do tempo. Os desafios e oportunidades tornam esta área, de desenvolvimento de software, um dos campos mais empolgantes hoje. Existem diversas teorias e estratégias para tornar os programadores mais produtivos e assertivos, algumas delas serão apresentadas neste material e outras só serão adquiridas com a experiência e o tempo. Não importa que tipo de trabalho você acabe fazendo, aprender as estratégias para o desenvolvimento de programas, conhecer o pensamento computacional e iniciar resolvendo problemas de baixa complexidade será importante em qualquer um dos trabalhos futuros. Para deseenvolver programas que solucionem os problemas, dos mais simples aos mais complexos é importante que sejamos capazes de prever o que esses programas farão, isso é mais fácil se entender como a linguagem de programação funciona e qual a importância dela no processo de desenvolvimento de software.

Os computadores entendem a linguagem de máquina (zeros e uns que na realidade são ligado e desligado) e os humanos se comunicam por sua linguagem natural. A linguagem de programação entra como uma linguagem intermediária entre a linguagem humana e a linguagem de máquina. Um programa é uma forma que facilita ao humano instruir e comunicar ao computador para fazer diferentes tarefas e ações. No final das contas os programas são transformados em linguagem de máquina. Código fonte é o programa escrito em linguagem de programação, enquanto o código executável é o programa transformado para a linguagem de máquina, também conhecida como linguagem binária. Normalmente a transformação do programa em executável segue dois caminhos, dependendo da linguagem de programação, teremos programas compilados onde a tarefa de transformar em executável é do compilador em tempo de programação e programas interpretados; com a geração do executável se dá no momento da execução do código por um programa chamado interpretador. Neste momento esses assuntos ainda complicam o entendimento, e podem ser deixados para o futuro. Uma questão importante para ser definida neste momento sobre linguagem de programação

é que neste e-book foi escolhida a Python, uma linguagem de programação lançada na década de 1990, mas extremamente atual, fácil de utilizar e versátil. Python pode ser utilizada para diversas aplicações e será útil para todos os exemplos deste material. A linguagem de programação Python será melhor apresentada na seção 1.4.

Figura 1 - Importância da Linguagem de Programação.

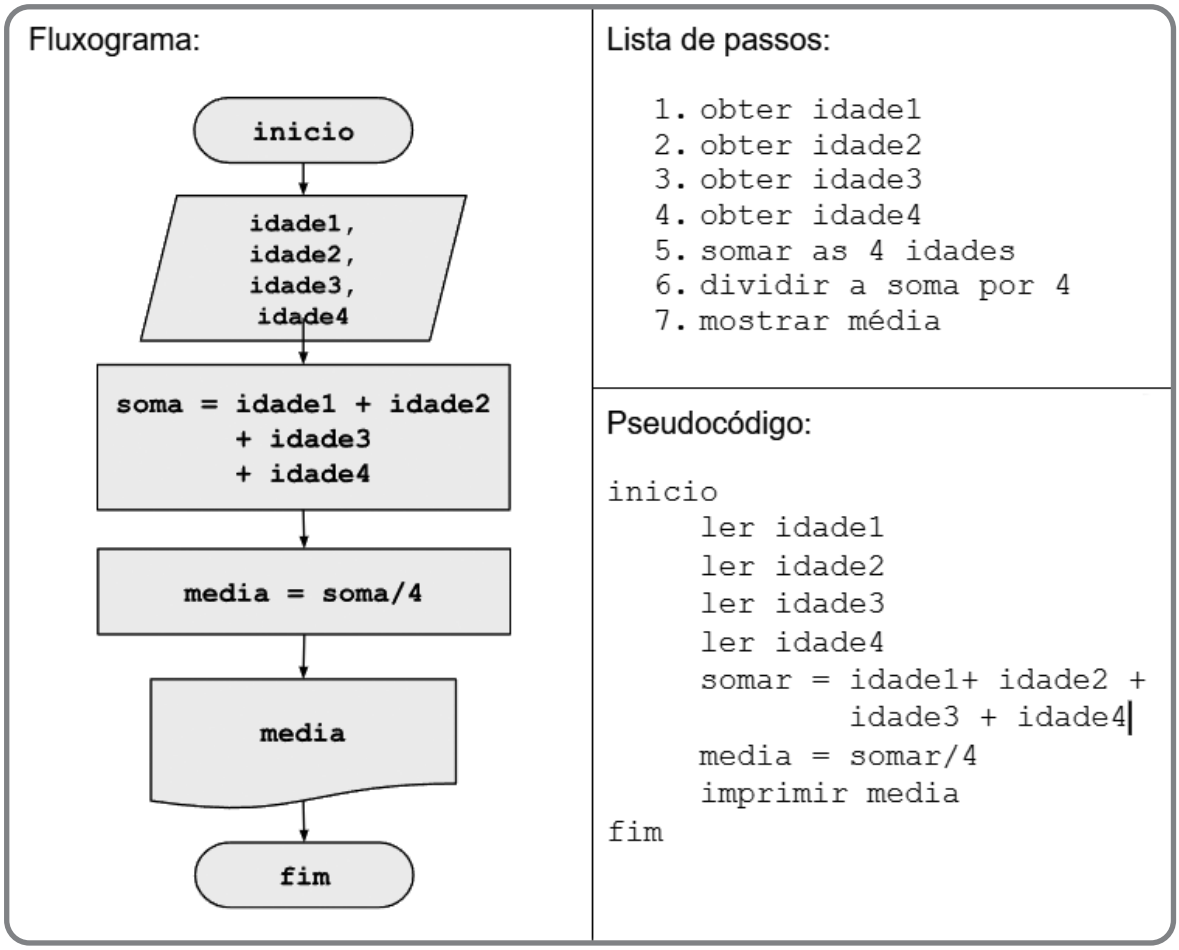


Fonte: autoria própria.

1.1.1 Algoritmos

Uma questão importante é a lógica computacional. Lógica é a ciência dos princípios formais do raciocínio. Refere-se ao domínio do pensamento, significa “saber pensar”. Para escrever um programa de computador, o raciocínio lógico é utilizado. A programação de computadores envolve o conceito de algoritmo, que é um conceito fundamental na programação. Algoritmo é o conjunto de passos para solucionar um problema utilizando o raciocínio lógico. No cotidiano é possível encontrar inúmeros exemplos de algoritmos para resolver problemas. Trocar o pneu de um carro, assar um churrasco, lavar a louça, executar um truque de mágica, montar um quebra cabeças, resolver um cubo mágico, entre outros. Os algoritmos podem ser representados através de fluxogramas, listas de passos em um documento ou com pseudocódigo (que é escrito de forma semelhante a código que deve ser desenvolvido). É importante salientar que não são necessárias as três, normalmente se escolhe uma delas. Na figura a seguir são apresentados os três modelos de representação de algoritmos para um problema de cálculo de média de idade de quatro pessoas.

Figura 2: Representações para algoritmos

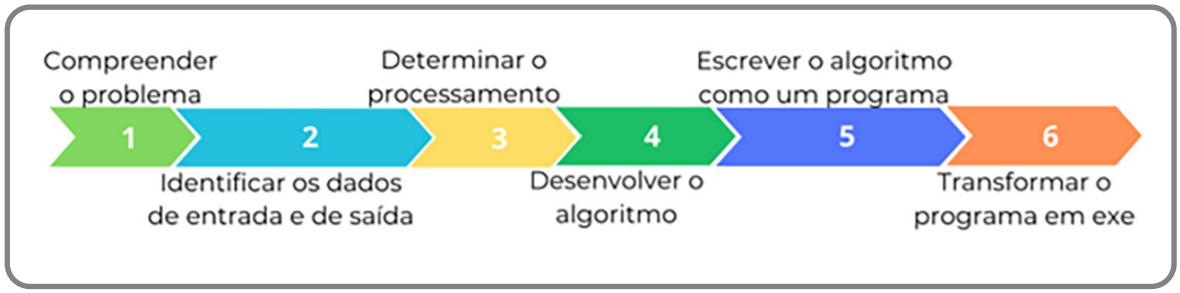


Fonte: autoria própria

Mesmo para a programação de computadores é possível criar algoritmos que deem o caminho inicial, mas é importante salientar que existem diversos algoritmos para solucionar o mesmo problema. Abaixo segue um dos algoritmos para criar um programa partindo de um algoritmo:

1. Compreender o problema.
2. Identificar os dados de entrada e de saída.
3. Determinar o que deve ser feito para transformar as entradas determinadas nas saídas especificadas - processamento.
4. Desenvolver o algoritmo.
5. Escrever o algoritmo como um programa.
6. Transformar o programa em executável (isso é feito por outro programa).

Figura - Estratégia para desenvolvimento de Algoritmos.



Fonte: autoria própria

1. Compreender o problema: Inicialmente é importante pensar um pouco sobre o que precisa ser resolvido. Neste momento a ideia é focar no que deve ser obtido, nas restrições existentes e não em como resolver o problema. Considere realizar uma série de perguntas associadas ao problema ou ao cálculo a ser realizado. Por exemplo: Qual é o objetivo buscado? Quais cálculos devem ser realizados?

2. Identificar os dados de entrada e de saída: Para identificar os dados de entrada, deve-se responder com relação ao problema as perguntas: quais valores ou informações precisam ser conhecidos para resolver o problema? Que valores ou informações mudariam para diferentes instâncias do problema? Por exemplo, o tamanho dos lados e altura do triângulo; quantidade de litros de combustível consumido e distância percorrida por um carro. Em relação aos dados de saída, qual o resultado esperado, como a área do triângulo e o consumo em quilômetros por litro do carro.

3. Determinar o processamento: Ao determinar o processamento, é importante observar regras e limitações. Identificar ações a serem realizadas como a fórmula da área do triângulo. Eliminar ambiguidades, por exemplo, é o usuário quem fornece os valores para calcular o consumo do carro?

4. Desenvolver o Algoritmo: Para construir o algoritmo, deve-se pensar os diversos passos para resolver o problema. Normalmente é possível resolver o problema de mais de uma maneira, ou seja, é possível ter mais de um algoritmo para resolver o mesmo problema.

5. **Escrever o Algoritmo:** Nesse momento o fluxograma ou as outras técnicas serão desenvolvidas.

6. Transformar em programa: O algoritmo ainda não é um programa, ele é uma forma avançada de descrição do problema, mas para transformar em algo que o computador entende é necessário transcrever o algoritmos para uma linguagem de programação. Durante o texto a linguagem de programação Python será utilizada para essa tarefa. Depois do programa implementado serão utilizados outros programas para

transformá-lo em linguagem de máquina e executável. São eles o compilador ou o Interpretador. Esse material não entra em detalhes sobre essas duas abordagens, porém sugere que o leitor busque mais informações. Para ilustrar essa etapa, segue a listagem de um script em Python desenvolvido a partir do algoritmo apresentado anteriormente de várias formas.

```
idade1 = int(input("Digite a 1a idade:"))

idade2 = int(input("Digite a 2a idade:"))

idade3 = int(input("Digite a 3a idade:"))

idade4 = int(input("Digite a 4a idade:"))

soma = idade1 + idade2 + idade3 + idade4

media = soma/4

print(f"A média entre as 4 idades é {media} anos.")
```

Para aprender a programar, é preciso construir algoritmos, muitos. É necessária perseverança. Não se aprende apenas lendo algoritmos. Na construção de algoritmos, sempre dá para melhorar! Testar é fundamental e uma ferramenta primordial para o teste é o chamado Teste de Mesa!

Existem centenas de conceitos básicos associados a programação, abaixo segue uma pequena amostra para que o leitor se ambiente com alguns termos que serão utilizados ao longo do texto.

- **Algoritmo**: é uma sequência finita de ações executáveis que visam obter uma solução para um determinado tipo de problema. Existe mais de um algoritmo para resolver o mesmo problema.
- **Atribuição**: o = é a instrução de atribuição, por exemplo em "x = 3.9 * x", a parte do lado direito de = é uma expressão matemática. O * é usado para indicar multiplicação, uma vez que o valor é calculado, ele é armazenado de volta (atribuído) em x. Exemplo: <variável> = <expressão>
- **Biblioteca (library)**: normalmente associada a um projeto externo, composto por um módulo, um pacote ou um conjunto de pacotes. São projetos de terceiros que estendem as funções da linguagem, por exemplo pandas, opencv e matplotlib. Para instalar bibliotecas Python, normalmente é utilizada a ferramenta de gestão

de ambientes pip ou um instalador próprio. A biblioteca padrão acompanha a instalação padrão Python. São consideradas parte da linguagem, por exemplo, math, time, string, sys, etc. (https://docs.python.org/pt-br/3/library/index.html) Para listar os módulos e pacotes que compõem a biblioteca padrão, digite em um terminal em um computador com Python instalado (por exemplo cmd no windows) as seguinte linha: pip list. Para saber mais sobre cada componente, utilize a função help(). Por exemplo >>>import math e depois >>>help(math)

- **Código Fonte**: O código-fonte (também chamado de **fonte** ou **código**) é uma versão do programa da forma em que ele foi originalmente escrito em texto puro.
- **Comentários**: As linhas que começam com # são chamadas de comentários, destinado a leitores humanos e ignorado pelo Python, pula o texto de # para a próxima linha. Para comentar mais de uma linha utilize “””” no início do trecho e mais “””” no final do trecho.
- **Depurador/Debugger:** Um depurador (em inglês: debugger) é um programa de computador usado para testar outros programas e fazer sua depuração, que consiste em encontrar os defeitos do programa.
- **Entrada e saída**: As informações fornecidas ao algoritmo (entrada) e as informações produzidas pelo algoritmo (saída).
- **Estruturas de controle**: Instruções que alteram o fluxo de execução do programa, como estruturas condicionais (if, else) e loops (for, while).
- **Expressões**: Os fragmentos de código que produzem ou calculam novos valores de dados são chamados de expressões. Expressões mais simples podem ser combinadas usando operadores matemáticos (+, -, *, /, **). Espaços são irrelevantes dentro de uma expressão. A precedência matemática normal se aplica nas expressões dos programas. Exemplo: ((x1–x2)/2*n)+(spam/k**3). Tipos de expressões: Lógicas - produzem como resultado um valor lógico True ou False, por exemplo 5 > 4 (True) e expressões aritméticas, que produzem como resultado um valor numérico, por exemplo 1+3 = 4.
- **Identificadores**: Nomes são dados a variáveis (fahrenheit); constantes (pi); funções ou módulos ou métodos (main, converter); classes; etc . Os nomes são chamados de identificadores. Não é permitido colocar espaços nesses identificadores. Em Python os identificadores são case sensitive, ou seja, maiúsculas diferentes de minúsculas (Spam é diferente de spam e diferente de spAm).

- **Indentação**: conceito utilizado na programação para organizar e estruturar o código fonte de forma clara e legível. Consiste em adicionar espaços ou tabulações no início de cada linha de código, a fim de criar uma hierarquia visual que facilita a compreensão do programa, geralmente com 4 espaços ou um <tab>.
- **Instrução de entrada**: O objetivo de uma instrução de entrada é obter a entrada do usuário e armazená-la em uma variável. <variável> = input(<prompt>). Primeiro, o prompt é impresso para o usuário. A função "input" espera que o usuário insira um valor e pressione <Enter>. A expressão que foi inserida é avaliada para transformá-la de uma sequência de caracteres em um valor (um número por exemplo). O valor é atribuído à variável.
- **Linguagem de Programação**: é uma linguagem escrita e formal que especifica um conjunto de instruções e regras usadas para gerar programas. Esses programas também são chamados de Software.
- **Operações**: Ações realizadas sobre variáveis, como adição, subtração, multiplicação, divisão, etc.
- **Operadores**: Símbolos que representam operações (por exemplo, + para adição, - para subtração).
- **Palavras Reservadas** (keywords): São identificadores que são partes das linguagens de programação e não podem ser utilizados. Exemplos de Python: and, del, for, is, raise, assert, if, elif, in, print, dentre outros.
- **Parâmetro**: Os valores que serão recebidos por uma função, também chamados de argumentos.
- **Programa**: Os programas são as sequências de instruções baseadas nos algoritmos e escritas em uma linguagem de programação. Podem ter objetos constantes (invariantes durante a execução do algoritmo) ou variáveis (valores que podem ser alterados).
- **Script:** No contexto de programação, um script é uma série de comandos escritos para serem executados por um programa de computador. Normalmente um script requer um interpretador enquanto um programa requer um compilador. No entanto, não há diferença na criação do código.
- **Sequência**: A ordem na qual as instruções do algoritmo são executadas.

- **Sintaxe**: Regras que definem a estrutura do código na linguagem de programação.
- **Tipos de Dados**: Categorias de dados que determinam o tipo de valor que uma variável pode armazenar (por exemplo, inteiros, floats, strings).
- **Variável**: Uma variável é um espaço da memória do computador, identificada por um nome e que pode receber valores de um determinado tipo. O nome permanece sempre o mesmo, mas o conteúdo pode ser alterado quantas vezes for necessário, são como uma caixa onde são colocados valores. Quando uma variável muda, o valor antigo é apagado e um novo é escrito. Variáveis podem mudar de valor tantas vezes quanto forem necessárias.

1.2 O Paradigma da Programação Estruturada

A Programação Estruturada é um paradigma de programação que visa melhorar a clareza e a eficiência do código por meio da utilização de estruturas de controle bem definidas. Esse paradigma foi desenvolvido para superar as limitações dos métodos de programação menos organizados e introduz práticas que facilitam a leitura, a manutenção e a depuração do código.

Algumas vantagens da Programação Estruturada são a clareza e legibilidade, onde o código é mais fácil de entender e seguir, o que facilita a manutenção e a colaboração. A modularidade, onde subprogramas permitem a reutilização de código e facilitam a atualização e o teste de partes individuais do programa. Facilidade de depuração, porque o uso de estruturas de controle claras e funções facilita a identificação e correção de erros. A organização promove um design mais lógico e organizado do código.

É comum que os programadores iniciam sua jornada de aprendizado por esse paradigma de programação, neste momento é importante focar no pensamento computacional e na melhora das habilidades de desenvolvimento de algoritmos, por isso, durante o início do aprendizado não são trabalhados elementos gráficos como janelas, botões, entre outros. Os programas produzirão os resultados necessários ao desenvolvimento das habilidades através de modo texto, sem muitas “perfumarias”. Com o decorrer da carreira, aqueles que desejarem focar nas interfaces e no front end, ou mesmo os curiosos deverão se aprofundar nestes quesitos. É importante salientar que neste material as interfaces gráficas não serão tratadas.

A programação estruturada utiliza três estruturas de controle principais, as sequências, a seleção e a repetição. A **sequência** refere-se à execução de instruções em uma ordem linear. O código é executado do início ao fim, linha por linha, sem elementos que mudem essa sequência.

```
print("Início")
varX = 10
print(varX)

#Saída após a execução:
Início
10
```

A **seleção** (condição ou decisão) permite que o programa tome decisões com base em condições. As principais construções para isso são if, elif e else. Essas estruturas são a base para a implementação da inteligência nos programas, visto que a partir delas os programas podem tomar decisões de qual processamento seguir.

```
idade = 18
if idade >= 18:
     print("Adulto")
else:
     print("Menor de idade")

#Saída após a execução:
Adulto
```

Já a **repetição** (ou laços): Permite a execução repetida de um bloco de código enquanto uma condição for verdadeira. Os principais laços são for e while. Importante salientar que muitas aplicações não seriam possíveis de resolver sem esta estrutura de controle.

```
for i in range(5):
    print(i)
#Saída após a execução:
0
1
2
3
4
```

```
cont = 0
while cont < 5:
    print(cont)
    cont += 1

#Saída após a execução:
0
1
2
3
4
```

Na programação estruturada, o código é dividido em subprogramas que encapsulam blocos de código relacionados. Isso melhora a organização e a reutilização do código. Os subprogramas são chamados de funções ou procedimentos (em algumas linguagens). As funções ajudam a dividir o problema em partes menores e mais manejáveis.

```
#criação da função nas 2 linhas abaixo

def saudacao(nome):

     print(f"Olá, {nome}!")


#utilização da função na linha abaixo

saudacao("Maria")

     #Saída após a execução:

Olá, Maria!
```

1.3 o Ambiente Integrado de Desenvolvimento

Os Ambientes de Desenvolvimento Integrado, comumente conhecidos como IDEs, desempenham um papel fundamental no processo de desenvolvimento de software. Eles são ferramentas que reúnem em um único espaço todas as funcionalidades necessárias para a programação, facilitando a vida dos programadores e aumentando a eficiência do desenvolvimento. A seguir, serão destacadas algumas das principais importâncias e facilidades que os IDEs oferecem.

Uma das maiores vantagens dos IDEs é a **integração de diversas ferramentas** em um único ambiente. Isso inclui editores de código, compiladores, depuradores e sistemas de controle de versão. Essa unificação permite que os desenvolvedores realizem tarefas complexas sem precisar alternar entre diferentes aplicativos, economizando tempo e reduzindo a chance de erros. Neste contexto é importante salientar que o uso de IDE não é obrigatório eu que é possível desenvolver código utilizando um editor de texto qualquer e um compilador ou interpretador da linguagem de programação utilizada, essa abordagem pode funcionar no início da carreira, no aprendizado, porém, quando as aplicações e códigos começarem a crescer e ficarem complexos demais, os IDEs tomam uma importância quase obrigatória.

Os IDEs modernos oferecem recursos avançados de autocompletar e sugestões inteligentes, que ajudam os programadores a escrever código mais rapidamente e com menos erros. Esses recursos analisam o contexto do código e sugerem automaticamente funções, variáveis e métodos relevantes, tornando o processo de codificação mais ágil e intuitivo. Uma ressalve que essa funcionalidade pode ser prejudicial ao aprendizado, pois a fixação dos comandos e detalhes do programa passa despercebida para o aprendiz. No início, até os erros de digitação (sintáticos) ajudam na evolução do aprendizado.

A depuração é uma etapa crítica no desenvolvimento de software. Os IDEs fornecem ferramentas robustas para identificar e corrigir erros no código. Com recursos como pontos de interrupção, inspeção de variáveis em tempo real e execução passo a passo, os desenvolvedores podem diagnosticar problemas rapidamente, economizando tempo e esforço.

Os IDEs também facilitam o gerenciamento de projetos, permitindo que os programadores organizem seus arquivos, bibliotecas e dependências de maneira eficiente. Isso é especialmente útil em projetos grandes ou colaborativos, onde a estrutura do código pode se tornar complexa. Com um bom gerenciamento, é possível manter a organização e a clareza do projeto. Novamente é feita uma ressalva que os programas de baixa complexidade dos exemplos do e-book não necessitam deste gerenciamento.

Muitos IDEs oferecem suporte a múltiplas linguagens de programação, permitindo que os desenvolvedores trabalhem em diferentes projetos sem precisar mudar de ferramenta. Isso é particularmente vantajoso em ambientes onde diversas tecnologias são utilizadas, pois promove uma maior flexibilidade. Visual Studio, IntelliJ, Eclipse, Pycharm são alguns exemplos. Porém, atualmente o programador pode utilizar soluções web que apresentam funcionalidades como o desenvolvimento simultâneo por várias pessoas e não necessitam de instalação local. A maioria dos IDEs populares contam com uma comunidade ativa que compartilha tutoriais, plugins e extensões. Essa troca de informações enriquece o aprendizado dos programadores e possibilita a personalização do ambiente conforme as necessidades individuais ou específicas do projeto.

Os IDEs são essenciais para otimizar o trabalho dos programadores. Ao proporcionar uma série de ferramentas integradas que facilitam desde a escrita até a depuração do código, os IDEs não apenas aumentam a produtividade, mas também contribuem para a qualidade final do software desenvolvido. Para qualquer programador, seja iniciante ou experiente, investir tempo na familiarização com um bom IDE é um passo crucial para se tornar mais eficiente e eficaz na arte da programação.

Nesse sentido vale uma menção ao IDLE (Integrated Development and Learning Environment) que é o IDE que vem com a instalação padrão do Python. Ele foi projetado principalmente para facilitar o aprendizado da linguagem, oferecendo uma interface amigável e recursos úteis para iniciantes. Normalmente será instalado juntamente com a instalação do interpretador baixado de python.org.

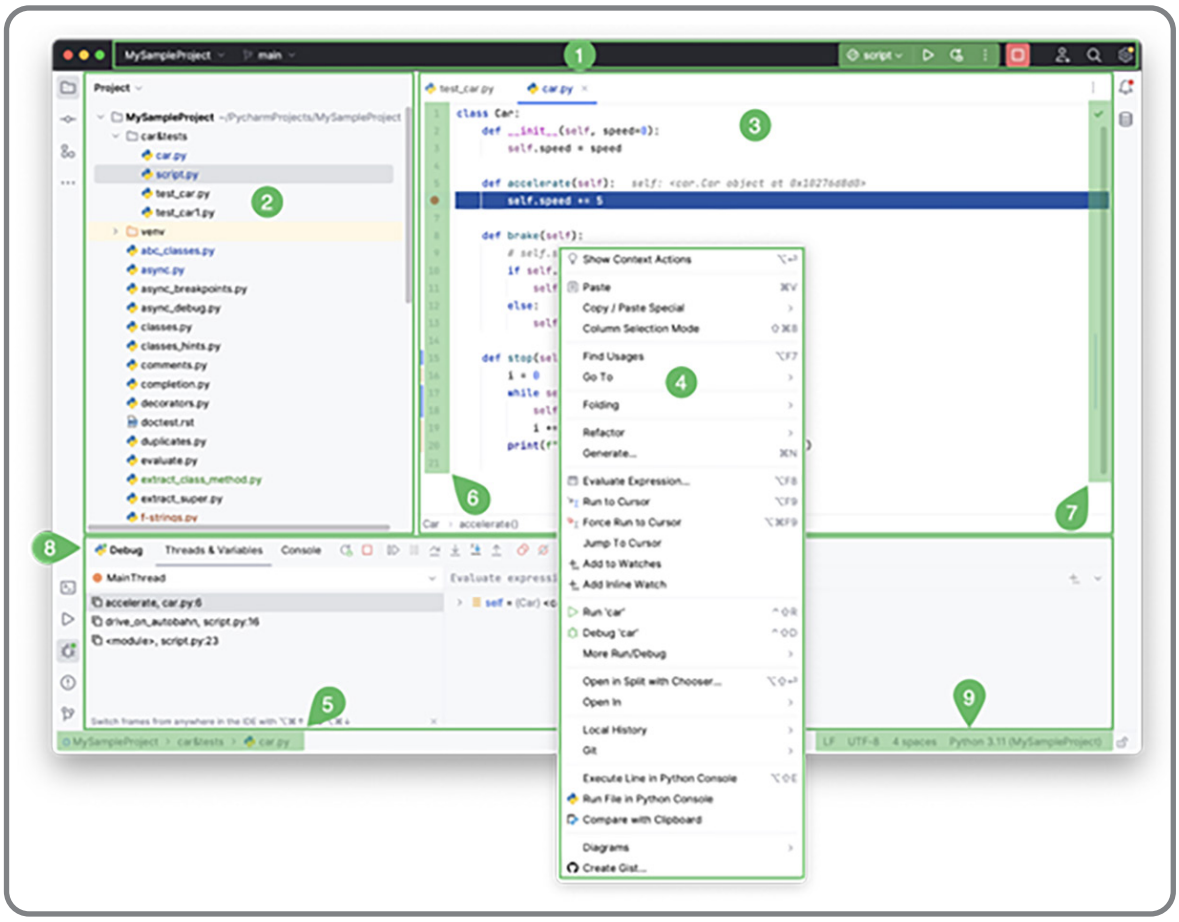
1.3.1 O IDE Pycharm

O PyCharm é uma das IDEs mais populares para desenvolvimento em Python, desenvolvida pela JetBrains (https://www.jetbrains.com/pycharm/). É amplamente reconhecido por sua especialização e recursos robustos que atendem tanto iniciantes quanto programadores experientes. O PyCharm se destaca pela sua Inteligência de Código, que inclui:

- Compreensão de Linguagem: A IDE entende a estrutura do Python, incluindo indentação, identificadores, classes e funções.
- Sugestões Automáticas: Enquanto o programador digita, o PyCharm sugere automaticamente a complementação de código, economizando tempo e reduzindo erros.
- Detecção de Erros em Tempo Real: A IDE verifica o código à medida que é escrito, destacando erros de sintaxe ou lógica antes da execução.

A interface do PyCharm é projetada para ser intuitiva, facilitando a navegação e a utilização das diversas ferramentas disponíveis. Isso permite que programadores se concentrem mais na codificação do que na configuração do ambiente.

Figura 3 - Interface gráfica do Pycharm



Fonte: jetbrains.com

1. O cabeçalho da janela contém um conjunto de componentes gráficos que fornecem acesso rápido às ações mais populares.
2. A janela de ferramentas do projeto no lado esquerdo exibe os arquivos do seu projeto.
3. Editor no lado direito, onde o código é escrito. Ele tem abas para navegação fácil entre arquivos abertos.
4. Os menus de contexto são abertos quando se clica com o botão direito do mouse em um elemento da interface ou em um fragmento de código e mostram as ações disponíveis.
5. A barra de navegação permite que se navegue rapidamente pelas pastas e arquivos do projeto.

6. Gutter, a faixa vertical ao lado do editor, mostra os pontos de interrupção e fornece uma maneira conveniente de navegar pela hierarquia de código, como ir para definição/declaração. Ela também mostra números de linha.

7. Barra de rolagem, no lado direito do editor. O PyCharm monitora constantemente a qualidade do código executando inspeções. O indicador no canto superior direito mostra o status geral das inspeções de código para o arquivo inteiro.

8. As janelas de ferramentas são janelas especializadas anexadas à parte inferior e às laterais do espaço de trabalho. Elas fornecem acesso a tarefas típicas, como gerenciamento de projetos, pesquisa e navegação de código-fonte, integração com sistemas de controle de versão, execução, teste, depuração e assim por diante.

9. A barra de status indica o status do seu projeto e de todo o IDE, e mostra vários avisos e mensagens informativas como codificação de arquivo, separador de linha, perfil de inspeção e assim por diante. Ela também fornece acesso rápido às configurações do interpretador Python.

O PyCharm oferece suporte excepcional a diversos frameworks e bibliotecas populares em Python, como o Django que tem Integração completa com funcionalidades como autocompletar para modelos e visualizações, além de suporte à depuração. Para Ciência de Dados que é uma área muito popular para programadores Python, esta IDE possui integração com bibliotecas como NumPy, Pandas e Matplotlib o que facilita a análise de dados e visualizações. Além disso, a IDE possui uma integração nativa com Git, permitindo gerenciar repositórios diretamente no ambiente. Isso inclui operações como criação de branches (criados para trabalhar com uma cópia do código sem modificar a versão existente), histórico de commits (são utilizados para auxiliar o controle do versionamento e as modificações de sistemas) e comparação de alterações, tudo através de uma interface gráfica amigável.

O PyCharm está disponível em duas versões, Community Edition que é gratuita e ideal para projetos menores ou para quem está começando. Professional Edition que é a versão paga que oferece recursos adicionais voltados para desenvolvimento web e suporte a frameworks avançados.

O PyCharm é uma ótima escolha para desenvolvedores Python, oferecendo um conjunto abrangente de ferramentas que aumentam a produtividade e melhoram a qualidade do código. Sua especialização em Python, combinada com suporte a frameworks populares e integração com sistemas de controle de versão, faz dele uma ferramenta essencial para qualquer programador que deseja maximizar sua eficiência no desenvolvimento de software. A versão community é a escolha para os alunos que

podem realizar a instalação de programas em seus computadores, para aqueles estudantes que não tem a possibilidade de instalar o IDE, na próxima seção será apresentada uma alternativa web.

1.3.2 O IDE Vscode.dev

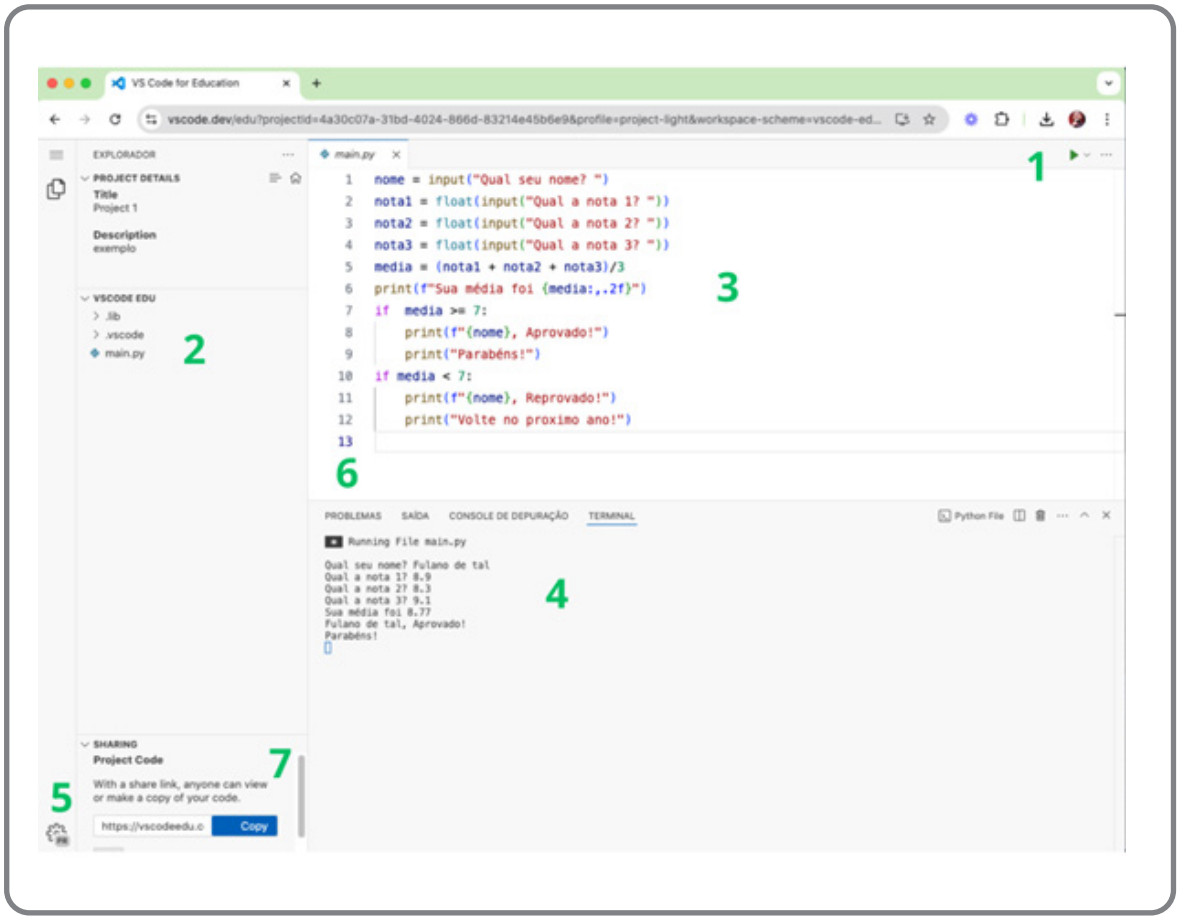
Uma alternativa web é o vscode.dev, também encontrado como vscodeedu.com é uma versão online do Visual Studio Code (VS Code), um dos editores de código mais populares desenvolvidos pela Microsoft. Essa versão permite que os usuários acessem um ambiente de desenvolvimento poderoso diretamente em seus navegadores, sem a necessidade de instalação local. O vscode.dev oferece a conveniência de um editor de código que pode ser acessado instantaneamente através de um navegador. Isso elimina a necessidade de instalação e configuração, permitindo que os desenvolvedores comecem a programar rapidamente, independentemente do sistema operacional.

Assim como sua contraparte desktop, o vscode.dev suporta uma ampla gama de linguagens de programação, incluindo JavaScript, Python, TypeScript, HTML, CSS e muitas outras. Isso torna a plataforma versátil para diferentes tipos de projetos e necessidades. O recurso IntelliSense, que fornece sugestões inteligentes e autocompletar para códigos, está presente no vscode.dev. Isso ajuda os desenvolvedores a escrever código mais rapidamente e com menos erros, oferecendo sugestões contextuais baseadas na linguagem utilizada, importante salientar o que já foi comentado anteriormente que essa funcionalidade pode atrapalhar o estudante.

Embora o vscode.dev não tenha todas as funcionalidades da versão instalada, ele ainda permite que os usuários compartilhem seus projetos facilmente. Isso é útil para colaboração em tempo real, onde várias pessoas podem trabalhar no mesmo código simultaneamente. O vscode.dev facilita o acesso a repositórios do GitHub diretamente no navegador. Os usuários podem abrir arquivos de projetos hospedados no GitHub e realizar edições rápidas sem sair da interface do editor. Novamente é importante que os desenvolvedores conheçam o GitHub e na medida do possível postem seus projetos lá.

A interface do vscode.dev mantém a simplicidade e a organização que caracterizam o VS Code desktop. Os desenvolvedores podem navegar facilmente entre arquivos e pastas, personalizando seu ambiente conforme necessário. O vscode.dev representa uma extensão prática das capacidades do Visual Studio Code, permitindo que desenvolvedores acessem um ambiente robusto de codificação diretamente no navegador. Com suporte a múltiplas linguagens, recursos avançados como IntelliSense e colaboração em tempo real, essa versão online é uma ferramenta valiosa tanto para iniciantes quanto para profissionais que buscam flexibilidade e eficiência em seus projetos de programação.

Figura 4 - Interface gráfica do IDE Vscode for education



Fonte: vscode.dev

1. Botão para execução dos programas e configuração do editor nas reticências.
2. A janela de ferramentas do projeto no lado esquerdo exibe os arquivos do seu projeto.
3. Editor onde o código é escrito. Ele tem abas para navegação fácil entre arquivos abertos.
4. Console com a execução e interação do código.
5. Ferramentas para configuração.
6. Faixa vertical ao lado do editor, mostra os pontos de interrupção e fornece uma maneira conveniente de navegar pela hierarquia de código. Ela também mostra números de linha.
7. Link para compartilhamento do código, onde várias pessoas podem trabalhar no mesmo código simultaneamente.

Pycharm e vscode.dev são algumas das alternativas de IDE que o estudante pode utilizar para o desenvolvimento dos exemplos e exercícios. Outras opções que dispõem de muitos tutoriais são o Google Colab e o Jupyter Notebook. É importante programar para aprender a programar. Comece copiando os exemplos e testando, mas em seguida desenvolva sua própria solução. Neste momento, uma estratégia importante é digitar todos os códigos. Isso mesmo, sem copiar e colar, digitar tudo, caracter por caracter. A digitação faz com que o estudante fixe os comandos digitados, isso é importante para o aprendizado. Se for possível, outra ação importante seria desabilitar a inteligência artificial do IDE. Antes de finalizar as unidades 2, 3 e 4 o estudante encontrará um conjunto de exercícios. É recomendado que complete os exercícios antes de continuar a leitura da próxima seção. Estes exercícios são parte do aprendizado e da fixação dos conceitos apresentados nas unidades.

1.4 A Linguagem de Programação Python

Python é seguramente a melhor linguagem para programadores iniciantes ou provindos de outras áreas (GUILHON, 2022)! Boas fontes para iniciantes podem ser encontradas em https://docs.python.org/pt-br/3/tutorial/index.html#tutorial-index e em https://docs.python.org/pt-br/3/reference/index.html#reference-index além deste ebook.

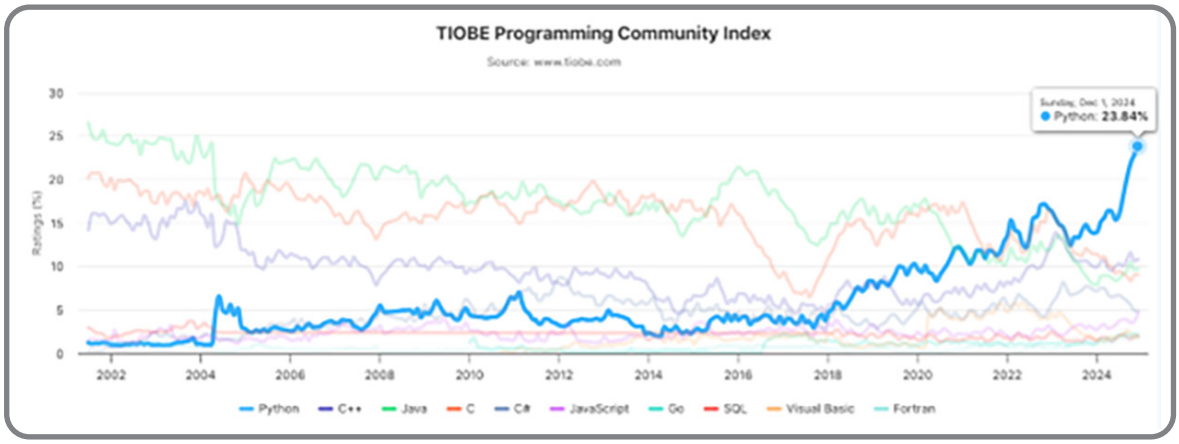
Python é uma das ferramentas mais populares e acessíveis para quem está começando a jornada na programação, mas também é a opção para programadores experientes. Python é uma linguagem de programação de alto nível, conhecida por sua sintaxe clara e concisa, que permite que os programadores se concentrem na resolução de problemas em vez de se perderem em complexidades técnicas. Ela é classificada como uma linguagem de script pelo caráter interpretado ao invés de compilada. Porém, é possível gerar executáveis a partir dos scripts Python. Ela pode ser utilizada em aplicações web, scripts para automatização de processos, desenvolvimento de jogos, Ciência de Dados, Internet das coisas, Visão computacional, dentre outras. No link https://www.python.org/about/success/ é possível encontrar uma listagem de histórias de sucesso de Python em projetos reais.

Python tem uma variedade de bibliotecas que facilitam o desenvolvimento. Dentre as quais é possível citar a NumPy: Para cálculos numéricos e manipulação de arrays; Pandas: Para análise e manipulação de dados; Matplotlib: Para visualização de dados; Flask e Django: Para desenvolvimento web e TensorFlow e PyTorch: Para aprendizado de máquina e inteligência artificial.

A comunidade Python é vasta e acolhedora. Existem inúmeros recursos disponíveis, como tutoriais, fóruns e bibliotecas, que podem ajudar a resolver problemas e expandir suas habilidades. Um ótimo começo é pelo portal https://www.python.org que conduzirá o estudante a diversos conteúdos sobre a linguagem Python, desde o interpretador mais atual para ser instalado, passando por documentação e até exercícios.

Python foi criada nos anos 90 por Guido Van Rossum, possui uma licença de código aberto e entre outras coisas permite que os programadores possam modificar seu código, utilizá-lo e redistribuí-lo livremente sem ter que pagar ao autor original. Python é a primeira linguagem do ranking Tiobe (2024), que lista as linguagens de programação mais utilizadas desde o ano 2000. Conforme a Figura a seguir, é possível acompanhar a evolução da linguagem nos últimos 24 anos.

Figura 5: Evolução do uso das principais linguagens de programação desde 2000



fonte: tiobe.

Dentre as principais características da linguagem, Python é uma linguagem interpretada, o que significa que pode executar o código linha por linha, facilitando a depuração e desenvolvimento. Tipagem Dinâmica: não precisa declarar o tipo de variável explicitamente. O Python determina o tipo automaticamente, o que torna o código mais flexível. A estrutura de controle (como loops e condicionais) é definida por indentação, o que contribui para a legibilidade do código. Um dos objetivos é que seja fácil de escrever os scripts, de uma forma próxima da escrita de texto em inglês. Veja a seguir o seguinte exemplo de um try-except:

```
try:
      #faz algo
except:
      #trata exceção
```

A linguagem é baseada em uma filosofia própria, baseada em algumas premissas de Python. Essas premissas conhecidas como Zen of Python (https://pt.wikipedia.org/wiki/Zen_de_Python), são listadas no quadro a seguir:

Quadro 1: O Zen do Python, de Tim Peters.

```
Bonito é melhor que feio.
Explícito é melhor que implícito.
Simples é melhor que complexo.
Complexo é melhor que complicado.
Plano é melhor que aninhado.
Esparso é melhor que denso.
A legibilidade conta.
Casos especiais não são especiais o suficiente para quebrar as regras.
Embora a praticidade supere a pureza.
Os erros nunca devem passar silenciosamente.
A menos que seja explicitamente silenciado.
Diante da ambiguidade, recuse a tentação de adivinhar.
Deveria haver uma -- e de preferência apenas uma -- maneira óbvia
 de fazer isso.
Embora essa forma possa não ser óbvia à primei-
ra vista, a menos que você seja holandês.
Agora é melhor do que nunca.
Embora nunca seja melhor do que *agora*.
Se a implementação for difícil de explicar, é uma má ideia.
Se a implementação for fácil de explicar, pode ser uma boa ideia.
Namespaces são uma ótima ideia -- vamos fazer mais deles!
```

Fonte: execute no IDE: import this

As variáveis são um dos elementos chave que se utiliza em Python e em muitas das linguagens de programação. Variáveis armazenam os valores dos programas. Para criar uma variável em Python, simplesmente é dado um nome (identificador) a ela e a associa a um valor usando o operador de atribuição =.

Por exemplo:

```
nome = "João"
idade = 30
altura = 1.75
```

Aqui nome, idade e altura são variáveis que armazenam, respectivamente, uma string (conjunto de caracteres), um inteiro e um número de ponto flutuante (conjunto dos números reais).

Para definir o nome da variável é importante estar atento às seguintes regras: devem começar com uma letra (a-z, A-Z) ou um underscore (_), podem ter um ou mais caracteres, podem conter letras, números (0-9) e underscores, não podem ter espaços em branco nem acentos, não podem ser palavras reservadas, não podem utilizar caracteres especiais, somente letras e números, não podem começar com um número. É importante que o nome apropriado a variável é crucial para o entendimento dos programas. Uma convenção importante em muitos livros e tutorias é a utilização do estilo de nome conhecido como Camelo (camelCase), que consiste na união de mais de uma palavra, sem espaço entre elas e com primeira letra maiúscula a partir da segunda palavra, por exemplo, mediaDeIdadeDosAnimais.

Os valores a serem usados podem ser dos tipos escalar ou simples: inteiro (números positivos ou negativos), real (números positivos, negativos e fracionários), lógico (verdadeiro ou falso) ou texto (caracteres) estruturados ou compostos: matriz, vetor, string, conjunto, registro ou arquivo. Durante a solução do problema são realizadas diversas ações sobre os valores (variáveis ou constantes) denominadas operações, tais como: soma (+), subtração (-), multiplicação (*) ou divisão (/). A tabela a seguir apresenta os tipos de dados e exemplos desses tipos. Para verificar o tipo de uma variável no código é possível utilizar a função type(). Por exemplo:

```
valor = 1
print(type(valor))

#Saída após a execução:
<class 'int'>
```

Tabela 1 - Tipos de dados Python

NOME	TIPO	EXEMPLOS
Inteiros	int	7, 900, 12312, -1
Reais	float	3.14, 2.7
Booleanos	bool	True, False
String	str	"Python", "jornada"
Listas	list	['danca','corrida','boxe']
Tuplas	tuple	(3, 7, 8)
Dicionários	dict	{'nome':'davi', 'jogo':'xadrez'}
Set	set	set([6, 9, 1])

Fonte: (GUILHON, 2022)

Python é uma linguagem de tipagem dinâmica, o que significa que você não precisa declarar o tipo da variável ao criá-la. O Python determina o tipo automaticamente com base no valor atribuído.

Por exemplo:

```
varX = 10  #varX é um inteiro
varX = "texto"   #agora varX é uma string
```

É possível usar a função type() para verificar o tipo de uma variável:

```
varA = 3.14
print(type(varA))
# Saída após a execução:
<class 'float'>

varB = "Olá"
print(type(varB))
# Saída após a execução:
<class 'str'>
```

O escopo de uma variável determina onde ela pode ser acessada. Em Python, você tem variáveis locais e globais. Variáveis Locais são definidas dentro de uma função e só podem ser usadas dentro dessa função. Variáveis Globais são definidas fora de todas as funções e podem ser usadas em qualquer parte do código.

A linguagem Python se encontra na terceira versão. O Python 2 foi atualizado até a versão 2.7, que atualmente só recebe correções, essa versão é mantida em função de sistemas que ainda utiliza. Todas as novas funcionalidades são lançadas no Python 3, por isso é a melhor escolha atualmente, pois existem algumas diferenças entre as versões. O clássico exemplo é o print, que no Python 2 tinha sintaxe sem parênteses. No python 3 print é uma função e precisa de parênteses. A versão 3.0 foi lançada em 2008. Para saber a versão instalada no seu computador digite em um terminal (cmd no windows): python --version ou python3 --version

1.4.1 Oi Mundo Python

É uma tradição na programação que o primeiro programa seja um "Oi mundo!", essa tradição já apresenta a função que será utilizada muitas vezes para mostrar mensagens na tela para o usuário. Nesse momento inicial é importante salientar que os usuários somos nós mesmos, as saídas de dados serão mensagens na tela e as entradas serão via teclado. Um script Python pode ser um único arquivo com a extensão .py ou vários arquivos referentes a aplicação na mesma ou em diversas pastas.

```
print("Oi mundo!")
#Saída após a execução:
Oi mundo
```

Sobre o código anterior, do "Oi Mundo" é possível executar em qualquer IDE para Python. A função print() pode receber vários parâmetros, como o texto que será impresso na tela, que é passado dentro dos parênteses e entre aspas. Alguns exemplos mais avançados serão apresentados nas seções seguintes. Neste exemplo também surge um comentário de código. Na Python comentando uma linha, ou parte dela, a partir do #. Para comentar um trecho de código com mais de uma linha, inicia com "''' e finalize com "''' (três aspas duplas).Além de print(), outra função utilizada frequentemente durante o aprendizado é a função input(), para entrada de dados pelo teclado. A seguir um pequeno exemplo que mostra como a função input() espera o usuário digitar algo e teclar <enter> para dar sequencia a execução do script:

```
varNome = input("Digite seu nome")
print(f"Seu nome é {varNome}")
```

SCRIPT PYTHON 10:

```
pi = 3.141592 #para mais curiosos, procure por exemplos do uso
de <import math> e math.pi
raio = float(input('Qual o raio? '))
V = (4/3)*pi*(raio**3) #raio**3 eleva ao cubo o raio
print('Volume da esfera = ',V)
```

ALGORITMO 11:

```
-obter o raio
-calcular area
-mostrar area
```

SCRIPT PYTHON 11:

```
import math
raio = float(input('digite o valor do raio: '))
print('área de uma superfície esférica é:', (4*math.pi)*(raio**2))
```

Síntese da Unidade

Aqui está um resumo dos principais temas abordados nas 5 seções desta Unidade, sobre uma Introdução a Algoritmos e Programação

1. Conceitos Básicos de Algoritmos
    - **Importância da Programação:** A programação de computadores exige uma base sólida em lógica e raciocínio lógico.
    - **Algoritmo:** Definido como um conjunto de passos para resolver problemas, podendo ser representado por fluxogramas, listas de passos ou pseudocódigo.
    - **Representação de Algoritmos:** Exemplos de representações para calcular a média de idades.
2. O Paradigma da Programação Estruturada
    - **Definição:** A Programação Estruturada visa melhorar a clareza e eficiência do código.
    - **Estruturas de Controle:** Principais estruturas incluem sequências, seleções (if, elif, else) e repetições (for, while).
    - **Funções:** Organização do código em subprogramas para melhorar a legibilidade e reutilização.

- 3. Ambiente Integrado de Desenvolvimento (IDE)
    - **Importância dos IDEs:** Ferramentas que integram editores, compiladores e depuradores, aumentando a eficiência do desenvolvimento.
    - **Recursos dos IDEs:** Autocompletar, depuração e gerenciamento de projetos.
    - **Exemplos de IDEs:** PyCharm e vscode.dev, com ênfase em suas funcionalidades e suporte a diversas linguagens.
- 4. Linguagem de Programação Python
    - **Escolha da Linguagem:** Python é apresentada como uma linguagem versátil, fácil e atual.
    - **Conceitos Básicos:** Definições de termos como código fonte, variáveis, operadores e estruturas de controle.

Esse resumo abrange os temas principais e fornece uma visão geral do conteúdo abordado na unidade.

Bibliografia

BANIN, S. L. **Python 3 - Conceitos e Aplicações - Uma abordagem didática.** Rio de Janeiro: Érica, 2018. E-book. p.Capa. ISBN 9788536530253. Disponível em: minhabiblioteca.com.br/reader/books/9788536530253/. Acesso em: 15 out 2024.

DEEPSEEK. Disponível em: https://deepseek.com/. Acesso em: 25 fev. 2025.

LAMBERT, K. A. **Fundamentos de Python:** primeiros programas. São Paulo: Cengage Learning Brasil, 2022. E-book. p.i. ISBN 9786555584301. Disponível em: minhabiblioteca.com.br/reader/books/9786555584301/. Acesso em: 15 out 2024.

NUMPY. Disponível em: https://numpy.org/. Acesso em: 25 fev. 2025.

SEVERANCE, C. R. **Python para Todos.** CreateSpace Independent Publishing Platform. 2016. Disponível em: https://do1.dr-chuck.com/pythonlearn/PT_br/pythonlearn.pdf. Acesso em: 15 out 2024.

RAMALHO, L. **Aprenda a Programar.** Creative Commons. Disponível em: https://wiki.python.org.br/. AprendaProgramar ou em . Acesso em: 14 fev 2025.

POE. Disponível em: https://poe.com/. Acesso em: 25 fev. 2025.

PYTHON. **Documentação.** Disponível em: https://www.python.org/. Acesso em: 25 fev. 2025.

WAINER, J. **Expressões Relacionais, Lógicas, Comandos Condicionais e Funções.** Disponível em: https://www.ic.unicamp.br/~wainer/cursos/1s2024/102/aula03.html. Acesso em: 25 fev. 2025.

Estruturas Básicas de Controle - Sequência

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Baseado na lógica da Programação Estruturada, apresentar questões básicas e a programação sequencial

Objetivos Específicos

- Introdução a execução de programas em Python.
- Introdução às variáveis em um programa.
- Apresentar operadores aritméticos, relacionais e lógicos;
- Apresentar operações de entrada e saída. Desenvolver exercícios de fixação.

Questões Contextuais

- Quais as formas de executar programas em Python?
- Como trabalhar com variáveis?
- Qual a importância dos operadores aritméticos, relacionais e lógicos?
- Porque print() e input() são tão importantes neste início de jornada?

Editora LaSalle

2.1 Executando Programas em Python

Quando um script Python é escrito e executado, o interpretador Python lê esse programa e executa as instruções que ele contém. O interpretador é uma camada de software entre o seu script e o hardware do computador. Quando o pacote Python é instalado no computador, ele gera uma série de componentes que contém, no mínimo, o interpretador e a biblioteca padrão. Os detalhes da instalação do interpretador vão sofrer variações de acordo com a plataforma, normalmente Windows, Linux ou Mac OS tem processos diferentes de instalação. Independente do sistema operacional, o ponto de partida para instalar o interpretador Python e buscar instruções de instalação é no portal python.org (não é a única fonte) no link downloads há direcionamento para a instalação em diversas plataformas além destas 3 citadas. O Python vem pré-instalado na maioria das distribuições Linux e está disponível como um pacote em todas as outras.

Para a execução dos exemplos e exercícios o estudante pode escolher o IDE que mais lhe agrade. Este ebook pressupõem que tenha acesso a um IDE e utilize a versão 3 do Python. Para alguns exemplos iniciais será utilizado um simples terminal com linha de comando em outros o IDLE (Python's Integrated Development and Learning Environment) como IDE, para mais informações sobre o IDLE verifique em https://docs.python.org/3.12/library/idle.html.

IDLE faz parte da instalação do Python, se procurar por ele, provavelmente aparecerá um para cada versão do Python que instalada no computador (assumindo que seja o Sistema Operacional Windows, em outras plataformas é um download separado). Então, é possível pressionar a tecla <Windows> no teclado, digitar IDLE e rolar para baixo até encontrar a versão desejada.

Nesse sentido, é possível considerar iniciar os estudos no IDLE e mudar no futuro para um IDE mais poderoso (e mais complexo também), como PyCharm ou VS Code, por exemplo. Alternativamente, é possível executar o IDLE a partir da linha de comando, no CMD ou na pesquisa do Windows. Por exemplo, se possuir as versões 3.11 e 3.13 instaladas e quiser executar o IDLE com 3.11, poderia usar a linha de comando (lançador ou launcher):

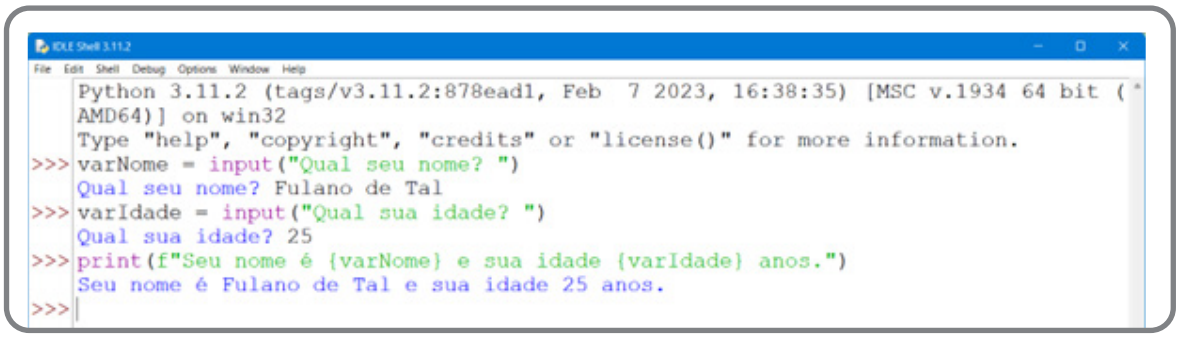
```
py -3.11 -m idlelib.idle
```

Considere o script python a seguir, além das 4 figuras na sequência que apresentam 3 formas diferentes para executar o mesmo programa, sendo escolhidas ao gosto ou disponibilidade do programador. A primeira Figura apresenta a execução com o ambiente interpretador IDLE, digitando e executando o script manualmente, linha a linha e pressionando a tecla <Enter> após cada linha. Na segunda Figura da sequência foi criado um arquivo Python com o script no editor do IDLE através de seu menu <File> e depois <New File>, onde a execução se é realizada posteriormente via menu

<Run> ou a tecla <F5>. A execução é apresentada da terceira figura, sendo este método o escolhido para executar os scripts deste livro. A quarta Figura apresenta a execução via linha de comando no prompt CMD do Windows, para essa execução o arquivo com a extensão <.py> deve ter sido editado previamente e salvo como texto.

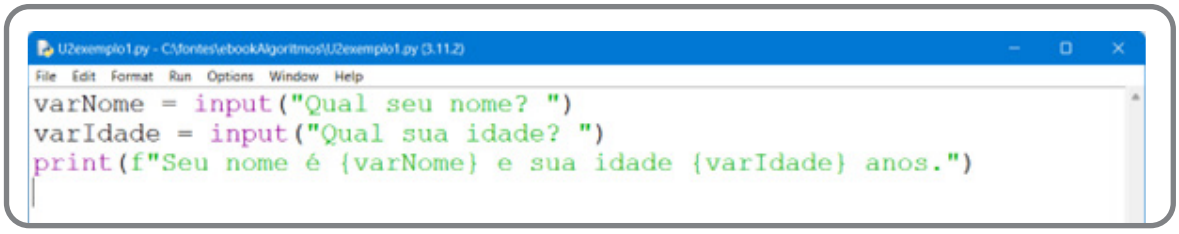
```
varNome = input("Qual seu nome? ")
varIdade = input("Qual sua idade? ")
print(f"Seu nome é {varNome} e sua idade {varIdade} anos.")
```

Figura 2.1 - IDLE editado e executando linha a linha



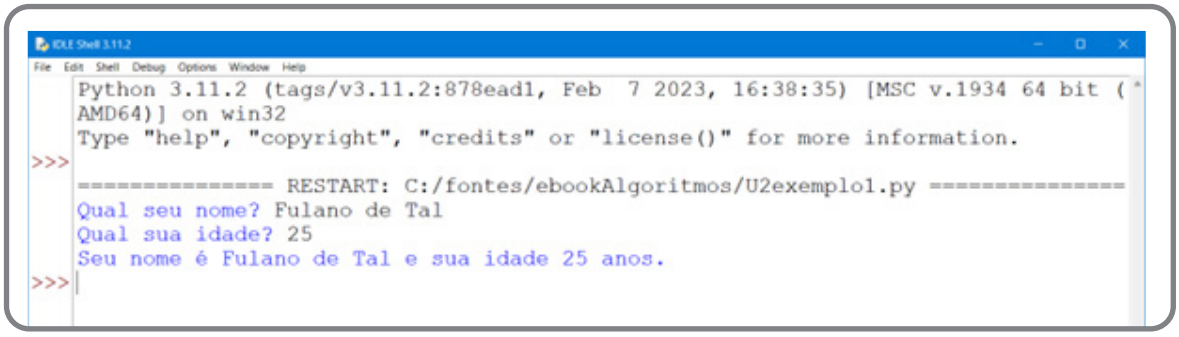
Fonte: própria autoria

Figura 2.2 - Editado no editor de texto do IDLE (para executar <F5>)



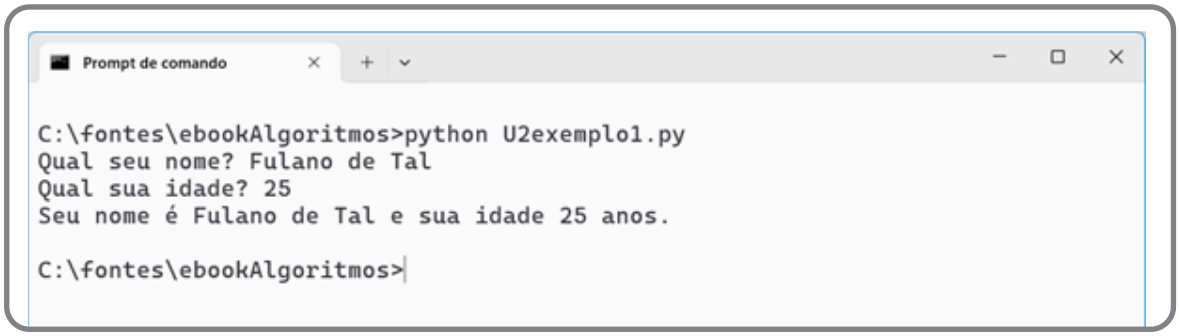
Fonte: própria autoria

Figura 2.3 - Resultado da execução no IDLE (depois de executar <F5>)



Fonte: própria autoria

Figura 2.4 - Execução do Script via linha de comando no terminal <cmd>



Fonte: própria autoria

Nesta fase do aprendizado ainda é cedo para entender a importância do "Teste de Mesa", o qual é uma técnica fundamental para o aprendiz de programador, especialmente na fase de desenvolvimento e depuração de código. Essa técnica envolve a simulação manual da execução de um algoritmo ou código, passo a passo, para entender como ele funciona e identificar possíveis erros analisando os valores das variáveis e fluxo do script a cada nova instrução. O Teste de Mesa ajuda a entender o código de forma mais profunda, antes mesmo de começar a usar o debugger. Ele te ensina a pensar como um programador, prevendo o comportamento do código e identificando erros antes que eles aconteçam.

No momento em que inicia o desenvolvimento das soluções para problemas de baixa complexidade é possível realizar o Teste de Mesa de forma manual. Com o crescimento da complexidade e mesmo do código, surge a necessidade de automatizar essa tarefa, para isso existem as ferramentas automáticas de depuração de código - debugger. O uso de um debugger complementa o teste de mesa. Enquanto o teste de mesa é uma abordagem manual e teórica, o debugger permite que o aprendiz veja a execução do código em tempo real, com a capacidade de pausar, inspecionar variáveis e seguir o fluxo de execução.

O teste de mesa é feito escolhendo valores para as variáveis de entrada, seguindo cada uma das etapas do algoritmo e verificando a correção da(s) saída(s). A seguir está um exemplo de um programa de baixa complexidade em Python que calcula a soma de dois números e imprime o resultado. Em seguida, é apresentado um teste de mesa para o código.

```
# Programa para somar dois números
# Entrada de dados
numero1 = int(input("Digite o primeiro número: "))
numero2 = int(input("Digite o segundo número: "))

# soma e impressão do resultado
resultado = numero1 + numero2
print(f"A soma é {resultado}")

#Saída após a execução digitando 5 e 3:
Digite o primeiro número: 32
Digite o segundo número: 15
A soma é 8
```

Para o teste de mesa serão consideradas as seguintes entradas e na sequência a tabela com o passo a passo do teste de mesa: numero1 = 5 e numero2 = 3

Tabela 2.1 - Teste de mesa para o código anterior

Passo	Descrição	Valor numero1	Valor numero2	Valor soma	Saída
1	Início do programa				Digite o primeiro número:
2	Usuário digita 5	5			
3	Entrada do segundo número	5			Digite o segundo número
4	Usuário digita 3	5	3		
5	Cálculo da soma	5	3	8	
6	Impressão do resultado	-	-	-	A soma é: 8

Fonte: própria autoria

O programa recebe dois números do usuário, calcula a soma e imprime o resultado. No exemplo, a soma de 5 e 3 resulta em 8. Este é um exemplo extremamente simples e pelo pequeno número de linhas talvez esse teste possa até ser realizado mentalmente, mas o objetivo aqui é justamente apresentar a técnica, para que o aprendiz

de programador aplique em problemas mais complexos que esse, por exemplo nos exercícios de fixação do final dos capítulos. Cabe ressaltar que os valores de entrada não foram validados, ou seja, caso sejam digitados valores que não sejam números, o programa gera um erro. Na seção 2.5 será apresentado um método para tratar validação de entrada através de try-except.

Ambas as técnicas — teste de mesa e debuggers — são essenciais para o aprendizado eficaz da programação. O teste de mesa solidifica a compreensão teórica, enquanto o debugger fornece uma abordagem prática e interativa que será usada por toda a carreira do programador. O teste de mesa foi apresentado no exemplo anterior, porém o exemplo da utilização do debugger será apresentado no decorrer das próximas seções.

2.2 Introdução ao Tratamento de Variáveis

As variáveis são um dos conceitos fundamentais na programação, representando locais na memória onde é possível armazenar dados. Em Python, as variáveis são dinâmicas, o que significa que não é necessário declarar seu tipo explicitamente, como por exemplo nas linguagens de programação C e Java. Em Python o interpretador determina o tipo do dado automaticamente, com base no valor atribuído.

Para declarar uma variável em Python, basta atribuir um valor a um nome de variável, esse nome também é chamado de identificador. Em Python, há bastante liberdade para definição do nome das variáveis, mas alguns cuidados podem ajudar a criar um código mais legível e fácil de entender.

Python é uma linguagem case-sensitive, o que significa que diferencia maiúsculas de minúsculas em nomes de variáveis, funções, classes e outros identificadores. Nesse exemplo a seguir, varNumero e VarNumero são duas variáveis distintas, o primeiro nome com v minúsculo e o segundo com V maiúsculo.

```
varNumero = 10
VarNumero = 20


print(varNumero)
print(VarNumero)


#Saída após a execução
10
20
```

O case sensitive também se aplica a funções, classes e importação de módulos, mas esses dois últimos casos - classes e módulos - serão vistos nas próximas unidades.

```
def minha _ funcao():
     return "Função com minúsculas"

def Minha _ Funcao():
     return "Função com maiúsculas"

print(minha _ funcao())
print(Minha _ Funcao())

#Saída após a execução
Função com minúsculas
Função com maiúsculas
```

A case-sensitivity permite que os desenvolvedores tenham mais liberdade na escolha de nomes e a capacidade de definir nomes semelhantes, mas distintos, para diferentes propósitos. No entanto, isso também significa que é importante ser consistente e cuidadoso ao nomear identificadores para evitar confusões e erros.

Pense em nomes que sejam descritivos, "idade" é melhor que "a" ou "x". Nomes que sejam curtos também facilitam, pois "nome_do_usuario" é melhor que "nome_completo_do_usuario_da_aplicacao". Outra questão é que o nome tenha um significado associado a função da variável, pois "valor_total" é melhor que "v", neste sentido é importante salientar que não convém usar nomes muito genéricos porque causam confusão, por exemplo "a", "b", "x", "y", "z". Embora em scripts pequenos criados durante o aprendizado possam aparecer esses nomes.

Essas regras de nomenclatura de variáveis descritas no parágrafo anterior são convenções, porém existem regras também. Não é possível utilizar palavras reservadas para nomes de variáveis, por exemplo "if", "else", "for", "while". As palavras reservadas no Python, ou em qualquer outra linguagem de programação, são palavras especiais que têm significados e propósitos específicos. No link https://docs.python.org/pt-br/3/reference/lexical_analysis.html#keywords é possível encontrar uma listagem com essas palavras reservadas. A seguir essas palavras são apresentadas em um quadro:

Quadro 2.1 - Palavras reservadas da linguagem Python

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Fonte: própria autoria

Uma convenção encontrada no desenvolvimento profissional é o uso do mesmo estilo de nomeação em todo o código. Por exemplo, usando snake_case, "nome_do_produto". Também é comum o camelCase onde o nome desta variável ficaria "nomeDoProduto". Nomes em inglês facilitam a comunicação com outros programadores. Um bom nome para uma variável ajuda a entender o código de forma rápida e intuitiva. A seguir alguns exemplos de criação de variáveis em Python:

```
nome = "Fulano"
idade = 25
altura = 1.74
```

Aqui, "nome" é uma variável do tipo str, "idade" é do tipo int e "altura" é do tipo float. Esses são os tipos de dados comuns encontrados na maioria dos programas. O tipo inteiros (int) é composto por números inteiros, positivos ou negativos. O tipo ponto flutuante (float) é composto por números com ponto decimal. Em portugues são utilizadas vírgulas nos números decimais, porém em inglês e nos programas é obrigatório números com pontos (por exemplo temperatura = 36.6). Já os dados tipo strings (str) são sequências de caracteres. Existem também os booleanos (bool) que representam valores de verdadeiro (True) ou falso (False).

Python é uma linguagem flexível em relação aos tipos de dados, mas ainda existem limites e considerações que os desenvolvedores devem ter em mente. Embora Python oferece uma grande flexibilidade em termos de tipos de dados e não imponha limites rígidos, as restrições de memória e os limites de precisão em tipos numéricos

e strings devem ser considerados ao desenvolver aplicações. É sempre bom monitorar o uso de memória e otimizar o código para evitar problemas de desempenho. A seguir, são listados os limites para os principais tipos de dados em Python.

Quadro 2.2 - Limites dos principais tipos de dados Python

Tipos Numéricos: **Inteiros (int):** Em Python 3, os inteiros são de precisão arbitrária, o que significa que você pode trabalhar com números inteiros muito grandes, limitados apenas pela memória disponível. Em versões anteriores, o tipo int tinha um limite fixo de 32 ou 64 bits. **Ponto flutuante (float):** O tipo float tem precisão limitada, normalmente seguindo a implementação do padrão IEEE 754 (https://pt.stackoverflow.com/ questions/493487/o-que-é-ieee-754), o que significa que a precisão é em torno de 15-17 dígitos decimais. Isso pode levar a erros de arredondamento.
Strings (str) As strings em Python são limitadas pelo tamanho da memória disponível. Não há um limite explícito para o comprimento de uma string, mas strings muito grandes podem levar a problemas de desempenho e uso excessivo de memória.

Fonte: própria autoria

Uma característica encontrada na linguagem Python é a possibilidade de atribuição de múltiplas variáveis em uma única linha, no exemplo a seguir, a variável varInt receberá o valor 1, a varFloat receberá o valor 2,5 e a varString receberá o valor "texto". As variáveis podem ser reatribuídas a qualquer momento, sendo que o novo valor substituirá o anterior.

```
varInt, varFloat, varString = 1, 2.5, "texto"
```

Em Python, assim como em qualquer outra linguagem de programação, é possível realizar operações matemáticas com as variáveis numéricas, assim como manipular as variáveis do tipo strings. Veja a seguir alguns exemplos de operações matemáticas seguidas de um exemplo de concatenação de string.

```
soma = idade + 5  # soma será 31
produto = idade * 2  # produto será 52
nome_completo = nome + " de Tal"
#nome_completo: "Fulano de Tal"
```

Python é considerada uma linguagem fortemente tipada. Isso significa que o tipo de dados de uma variável é rigidamente definido e não pode ser implicitamente convertido para outro tipo sem uma conversão explícita. Algumas razões que explicam essa característica. Python verifica tipos em Tempo de Execução: Em Python, o tipo de uma variável é verificado em tempo de execução. Se você tentar realizar operações entre tipos incompatíveis, um erro será gerado.

```
numero1 = 5
numero2 = "10"
soma = numero1 + numero2  # Isso gerará um TypeError
```

Outra razão está associada à conversão explícita necessária para misturar diferentes tipos, você precisa converter explicitamente um tipo em outro. Por exemplo, se você quiser somar um inteiro e uma string, deve converter a string em um inteiro primeiro.

```
a = 5
b = "10"
c = a + int(b)  # Isso funciona, pois a string foi convertida
```

Outra razão está associada a consistência de tipos, pois a forte tipagem ajuda a evitar erros comuns que podem ocorrer em linguagens de tipagem mais fraca, onde a conversão automática de tipos pode levar a comportamentos inesperados.

A questão de tipos de dados imutáveis também justifica. Muitos tipos de dados em Python, como strings e tuplas, são imutáveis. Isso significa que, uma vez que um objeto é criado, seu tipo e valor não podem ser alterados, reforçando a ideia de forte tipagem. Essas características tornam Python uma linguagem segura em termos de manipulação de tipos e ajudam a evitar muitos tipos de erros de programação.

Python é uma linguagem de programação que utiliza tipagem dinâmica, o que significa que você não precisa declarar o tipo de uma variável ao criá-la, como já foi

citado anteriormente nesta seção. O tipo é determinado em tempo de execução com base no valor que a variável recebe. A seguir são apresentadas cinco maneiras de como Python lida com a tipagem dinâmica:

1. Atribuição de valores sem declaração de tipo. Em Python, é possível atribuir qualquer tipo de valor a uma variável sem especificar seu tipo.

```
varExemplo = 10          # varExemplo é um inteiro
varExemplo = "Olá"       # varExemplo agora é uma string
varExemplo = 3.14        # varExemplo agora é um float
```

2. Mudança de tipo durante a execução. É possível mudar o tipo de uma variável a qualquer momento ao atribuir um novo valor.

```
varExemplo = [1, 2, 3]   # varExemplo é uma lista
varExemplo = (4, 5, 6)   # varExemplo agora é uma tupla
# nas próximas unidades serão explicadas listas e tupla
```

3. Funções e métodos com tipos variáveis. Funções em Python podem aceitar argumentos de diferentes tipos, permitindo uma grande flexibilidade.

```
def imprimir(valorRecebido):
     print(valorRecebido)

imprimir(10)          # Imprime um inteiro
imprimir("Texto")     # Imprime uma string
imprimir([1, 2, 3]) # Imprime uma lista

#saída após a execução
10
Texto
[1, 2, 3]
```

4. Verificação de tipo em tempo de execução. É possível usar a função type() para verificar o tipo de uma variável em tempo de execução.

```
varExemplo = 42
print(type(varExemplo))
#Saída após a execução:
<class 'int'>

varExemplo = "Python"
print(type(varExemplo))
#Saída após a execução:
<class 'str'>
```

5. Uso de anotações de tipo *(Type Hints*). A partir do Python 3.5, é possível usar anotações de tipo para sugerir tipos para variáveis e funções, mas isso é opcional e não altera a dinâmica da linguagem. Essas anotações ajudam na documentação e podem ser utilizadas por ferramentas de análise estática, mas não impõem restrições de tipo em tempo de execução.

```
def soma(varExemplo: int, varExemplo2: int) -> int:
     return varExemplo + varExemplo2
```

A tipagem dinâmica em Python permite que os desenvolvedores escrevam código de forma mais flexível e concisa. No entanto, isso também significa que os erros relacionados a tipos podem se manifestar em tempo de execução, exigindo cuidado durante o desenvolvimento e os testes da aplicação. O tratamento de variáveis em Python é simples e intuitivo. Compreender como declarar, atribuir e manipular

variáveis é essencial para o desenvolvimento de qualquer programa. À medida que avança o aprendizado de Python, a prática com variáveis se tornará cada vez mais natural e fundamental para resolver problemas e construir aplicações.

2.3 Operadores Aritméticos

Os operadores aritméticos em Python são utilizados para realizar operações matemáticas básicas. A seguir estão listados em um quadro os principais operadores aritméticos disponíveis na linguagem. Importante salientar que estes operadores trabalham da mesma forma que na maioria das outras linguagens. Esses operadores podem ser combinados de diversas maneiras para resolver problemas matemáticos e lógicos.

Quadro 2.2 - Operadores aritméticos em Python.

OPERADOR ARITMÉTICO	AÇÃO
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
//	Divisão inteira (quociente)
%	Módulo (resto da divisão)
**	Exponenciação

Fonte: própria autoria.

O operador de adição (+) é utilizado para somar dois ou mais números. Exemplo:

```
resultado = 4 + 7  # resultado será 11
```

O operador de subtração (-) é utilizado para subtrair um número de outro. Exemplo:

```
resultado = 20 - 7  # resultado será 13
```

O operador de multiplicação (*) é utilizado para multiplicar dois ou mais números.

```
resultado = 5 * 4  # resultado será 20
```

O operador de divisão (/) é utilizado para dividir um número por outro. O resultado será sempre um número de ponto flutuante (float).

```
resultado = 21 / 3  # resultado será 7.0
```

O operador de divisão inteira (//) retorna a parte inteira da divisão, descartando a parte decimal. Ele retorna o quociente da divisão.

```
resultado = 20 // 3  # resultado será 6
```

O operador de módulo (%) retorna o resto da divisão entre dois números. Exemplo:

```
resultado = 20 % 3  # resultado será 2
```

O operador de exponenciação (**) é utilizado para elevar um número a uma potência. Exemplo:

```
resultado = 23 ** 3  # resultado será 12167
```

Para testar todos os operadores, a seguir é apresentado um script Python comentado, com exemplos da utilização dos operadores aritméticos.

```
valor1 = 30
valor2 = 7
soma = valor1 + valor2                  # Adição
subtracao = valor1 - valor2             # Subtração
multiplicacao = valor1 * valor2         # Multiplicação
divisao = valor1 / valor2               # Divisão
divisao_inteira = valor1 // valor2      # Divisão inteira
modulo = valor1 % valor2                # Módulo
exponenciacao = valor1 ** valor2        # Exponenciação
print(f"Soma: {soma}")
print(f"Subtração: {subtracao}")
print(f"Multiplicação: {multiplicacao}")
print(f"Divisão: {divisao}")
print(f"Divisão Inteira: {divisao_inteira}")
print(f"Módulo: {modulo}")
print(f"Exponenciação: {exponenciacao}")
```

```
#Saída após a execução:
Soma: 37
Subtração: 23
Multiplicação: 210
Divisão: 4.285714285714286
Divisão Inteira: 4
Módulo: 2
Exponenciação: 21870000000
```

2.4 Operadores Relacionais e Lógicos

Os operadores relacionais são utilizados para comparar valores e retornar um valor booleano (True ou False). No termo operador relacional, relacional refere-se às relações que os valores podem ter uns com os outros. Eles são essenciais em estruturas de controle, como condicionais e loops. Os principais operadores relacionais em Python são:

Quadro 2.3 - Operadores relacionais e lógicos.

OPERADOR ARITMÉTICO	AÇÃO
==	igual
!=	diferente
>	maior
<	menor
>=	maior ou igual
<=	menor ou igual
**	Exponenciação

Fonte: própria autoria.

O operador relacional igual a (==) compara se dois valores são iguais. Exemplo:

```
resultado = (5 == 5)  # resultado será True
```

O operador relacional diferente de (!=) compara se dois valores são diferentes. Exemplo:

```
resultado = (5 != 3)  # resultado será True
```

O operador relacional maior que (>) verifica se o valor à esquerda é maior que o valor à direita. Exemplo:

```
resultado = (7 > 4)  # resultado será True
```

O operador relacional menor que (<) verifica se o valor à esquerda é menor que o valor à direita. Exemplo:

```
resultado = (3 < 5)  # resultado será True
```

O operador relacional maior ou igual a (>=) verifica se o valor à esquerda é maior ou igual ao valor à direita. Exemplo:

```
resultado = (5 >= 5)  # resultado será True
```

O operador relacional menor ou igual a (<=) verifica se o valor à esquerda é menor ou igual ao valor à direita. Exemplo:

```
resultado = (4 <= 6)  # resultado será True
```

Os operadores lógicos são utilizados para combinar expressões booleanas. Eles também retornam um valor booleano e são essenciais em estruturas de controle. Os principais operadores lógicos em Python são:

Quadro 2.4 - Operadores lógicos/expressões booleanas.

OPERADOR ARITMÉTICO	AÇÃO
and	e lógico
or	ou lógico
not	não lógico

Fonte: própria autoria

O operador lógico E lógico (and) retorna True se ambas as expressões forem verdadeiras. Exemplo:

```
resultado = (5 > 3) and (8 > 6)  # resultado será True
```

O operador lógico Ou lógico (or) retorna True se pelo menos uma das expressões for verdadeira. Exemplo:

```
resultado = (5 > 3) or (2 > 6)  # resultado será True
```

O operador lógico Não lógico (not) inverte o valor booleano da expressão. Exemplo:

```
resultado = not(5 > 3)  # resultado será False
```

Aqui está um exemplo prático que demonstra o uso de operadores relacionais e lógicos, para entender melhor seu funcionamento é importante executar o seguinte script Python modificando os valores das variáveis a e b, testando com valores diferentes e iguais para elas:

```
a = 10
b = 5

# Operadores Relacionais
igual = (a == b)                # False
diferente = (a != b)            # True
maior = (a > b)                 # True
menor = (a < b)                 # False
```

```
# Operadores Lógicos
condicao1 = (a > 0)                    # True
condicao2 = (b < 10)                   # True
resultado_and = condicao1 and condicao2   # True
resultado_or = condicao1 or (b > 10)      # True
resultado_not = not (a < b)               # True

print(f”Igual: {igual}”)
print(f”Diferente: {diferente}”)
print(f”Maior: {maior}”)
print(f”Menor: {menor}”)
print(f”Resultado AND: {resultado_and}”)
print(f”Resultado OR: {resultado_or}”)
print(f”Resultado NOT: {resultado_not}”)

#Saída após a execução:
Igual: False
Diferente: True
Maior: True
Menor: False
Resultado AND: True
Resultado OR: True
Resultado NOT: True
```

Os operadores relacionais e lógicos são fundamentais para a tomada de decisões em programação. Eles permitem que você compare valores e combine condições, facilitando o controle do fluxo de execução do seu código.

2.5 Operações de Entrada e Saída de Dados

Uma das funções mais importantes na fase de aprendizado na programação é a input(), a qual é utilizada para receber dados do usuário e armazená-los em variáveis. Importante salientar que a função input(), muito utilizada para entrada de dados retorna valores do tipo string, programador deve ficar atento a isso quando for ler um número, por exemplo um inteiro, esse dado vem da função input() como string, para utilizá-lo em cálculos é necessário utilizar a função int() que o transforma em inteiro. A função input é considerada uma função built-in da linguagem Python, ou seja, não é preciso fazer nenhum tipo de importação ou instalação para usá-la.

No início do aprendizado em programação, a entrada de dados se dará exclusivamente via teclado, mesmo sabendo que existem outras formas de inserir dados no computador como touch screen, mouse e webcam, por enquanto o teclado será o dispositivo de entrada. Para usar a função input, é possível escrever x = input() no código. Ao executar a instrução, o código irá pausar na linha do input, aguardando a entrada de dados do usuário. Ao apertar a tecla Enter, a entrada do usuário será transferida para a variável x (na forma de uma string). Para ilustrar o uso, as linhas abaixo solicitam o nome, a idade e a altura do usuário através da função input():

```
nome = input("Qual é o seu nome? ")
idade = int(input("Qual é a sua idade? ")) #Converte para int
altura = float(input("Qual é a altura?")) #Converte para float
```

No início do aprendizado, o próprio aprendiz será o usuário de seus programas, portanto, pressupõe-se que digite os valores esperados. Ou seja, se o script solicita um inteiro, será digitado um inteiro. Mas, caso precise ter certeza que não foi digitado valores diferentes dos solicitados, há como validar um input em Python. Para evitar erros de validação no input, é possível usar um bloco try-except da seguinte forma:

```
try:
    valorInt = int(input("Digite um inteiro: "))
    print(f'O número é {valorInt}.')
except ValueError:
    print("Atenção: Não foi digitado um número válido!")
```

O exemplo acima garante que a variável valorInt, lida pela função input(), seja um número inteiro. Caso contrário, a conversão para inteiro falhará e o código cairá no bloco except. Mesmo um número float gera exceção. Cabe repetir que no inicio, o aprendiz não precisa se preocupar com isso.

Outra função importante para o aprendizado, que é bastante utilizada é a função print(), a funcionalidade dela é exibir o conteúdo das variáveis. Essa será a principal função de saída de dados. A seguir é apresentada a sintaxe para escrever a função:

```
print(objeto,...,sep='   ',end='\n',file=sys.stdout,flush=False)
```

Neste exemplo, **objeto** refere-se ao que será impresso. O **sep** é o separador entre os objetos, o padrão é um espaço. O **end** é o que será impresso no final da linha, o padrão é uma quebra para a próxima linha, caso **não** queira que a linha seja quebrada para uma nova linha, utilizar end = ' '. Exemplo:

```
print("Bem-vindo ao programa!")
print("Resultado da soma:", 5 + 3)
#Saída após a execução:
Bem-vindo ao programa!
Resultado da soma: 8
#porém
print("Bem-vindo ao programa! ", end="")
print("Resultado da soma:", 5 + 3)
#Saída após a execução:
Bem-vindo ao programa! Resultado da soma: 8
```

A função print também tem o parâmetro sep=' ', que é um caractere de separação (por padrão, é um espaço em branco, caso seja omitido). Então, se for passado mais de um objeto para imprimir (string, int, etc), ele intercala os objetos com o caractere de separação. A seguir é apresentado um exemplo:

```
print('Oi','Sr.Programador')
#Saída após a execução:
Oi Sr.Programador
#porém
```

```
print('Oi', 'Sr.Programador', sep='@')
#Saída após a execução:
Oi@Sr.Programador
#ou
print('Oi', 'Sr.Programador','tudo','bem','?', sep='-oOo-')
#Saída após a execução:
Oi-oOo-Sr.Programador-oOo-tudo-oOo-bem-oOo-?
```

Para o exemplo a seguir, se for digitado "Fulano de Tal" para nome e 35 para idade no exemplo anterior da função input(), a saída desse print a seguir será: "Olá, Fulano de Tal! Você tem 35 anos.". Neste exemplo entre os parênteses foi iniciado com f, antes de abrir as aspas, para que seja possível dentro dessa string escrever expressões Python entre chaves, que podem se referir a variáveis, neste exemplo as variáveis nome e idade.

```
print(f"Olá, {nome}! Você tem {idade} anos.")
#Saída após a execução:
Olá, Fulano de Tal! Você tem 35 anos.
```

A forma mais simples e moderna de inserir variáveis em strings é utilizando f antes das aspas, como apresentado na linha anterior, que aceitam a inserção direta de variáveis no meio de um string usando chaves ({ }). Além dessa, outra forma possível e que será encontrada em muitos exemplos é através dos operadores mais (+) e/ou vírgulas (,). O operador mais (+) é utilizado para concatenar as strings e as vírgulas (,) para adicionar valores das variáveis a string do print. Dessa forma, também é possível inserir valores das variáveis no meio da string. A seguir é apresentada a linha do exemplo anterior, mas desta segunda forma.

```
print("Olá " + nome + "! Você tem", idade , "anos.")
#Saída após a execução:
Olá Fulano de Tal! Você tem 35 anos.
```

Síntese da Unidade

Aqui está um resumo dos principais temas abordados nas 6 seções desta Unidade, sobre as Estruturas básicas de controle - Sequência.

1. Executando Programas em Python

- Apresenta as ferramentas para iniciar a execução de um script Python.
- Dá algumas dicas sobre a execução dos scripts.
- Fala sobre a importância do teste de mesa.

2. Introdução ao Tratamento de Variáveis

- Definição de variáveis.
- Principais tipos de dados.

3. Operadores Aritméticos

- Apresenta os operadores aritméticos utilizados em Python.

4. Operadores Relacionais e Lógicos

- Apresenta os operadores relacionais.
- Apresenta os operadores lógicos.

5. Operações de Entrada e Saída De Dados

- Apresenta a função input().
- Apresenta a função print().

Esse resumo abrange os temas principais e fornece uma visão geral do conteúdo abordado na unidade

Bibliografia

BANIN, S. L. **Python 3 - Conceitos e Aplicações - Uma abordagem didática**. Rio de Janeiro: Érica, 2018. E-book. p.Capa. ISBN 9788536530253. Disponível em: minhabiblioteca.com.br/reader/books/9788536530253/. Acesso em: 15 out 2024.

DEEPSEEK. Disponível em: https://deepseek.com/. Acesso em: 25 fev. 2025.

LAMBERT, K. A. **Fundamentos de Python:** primeiros programas. São Paulo: Cengage Learning Brasil, 2022. E-book. p.i. ISBN 9786555584301. Disponível em: minhabiblioteca.com.br/reader/books/9786555584301/. Acesso em: 15 out 2024.

NUMPY. Disponível em: https://numpy.org/. Acesso em: 25 fev. 2025.

SEVERANCE, C. R. **Python para Todos**. CreateSpace Independent Publishing Platform. 2016. Disponível em: https://do1.dr-chuck.com/pythonlearn/PT_br/pythonlearn.pdf. Acesso em: 15 out 2024.

RAMALHO, L. **Aprenda a Programar.** Creative Commons. Disponível em: https://wiki.python.org.br/AprendaProgramar ou em . Acesso em: 14 fev 2025.

POE. Disponível em: https://poe.com/. Acesso em: 25 fev. 2025.

PYTHON. **Documentação**. Disponível em: https://www.python.org/. Acesso em: 25 fev. 2025.

WAINER, J. **Expressões Relacionais, Lógicas, Comandos Condicionais e Funções.** Disponível em: https://www.ic.unicamp.br/~wainer/cursos/1s2024/102/aula03.html. Acesso em: 25 fev. 2025.

Estruturas de Controle - Seleção e Repetição

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Baseado na lógica da Programação Estruturada, apresentar as estruturas de controle e de repetição

Objetivos Específicos

- Apresentar estruturas de controle.
- Apresentar estruturas de repetição.
- Princípios de vetores e matrizes.
- Desenvolver exercícios de fixação.

Questões Contextuais

- Qual a importância de estruturas de controle?
- Como trabalhar com estruturas de repetição?
- Qual a importância de vetores e matrizes?

Editora LaSalle

3.1 IF, ELIF E ELSE

A primitiva if (se) em Python é uma estrutura de controle que permite a execução condicional de blocos de código. Ela avalia uma expressão booleana e, dependendo do resultado TRUE ou FALSE (verdadeiro ou falso), decide qual bloco de código será executado. A estrutura if permite que o programa tome decisões com base em condições. Esta seção apresenta uma visão detalhada sobre o uso do if em Python, importante salientar que esta estrutura está presente em praticamente todas as linguagens de programação. A sintaxe e forma mais simples do if é a seguinte:

```
if condição:
   <bloco de código>
     # após a condição há : (dois pontos) e quebra de linha
     # condição: Uma expressão que pode ser avaliada como True ou False
     # bloco de código é o código que será executado se a condição for verdadeira. O bloco de código deve estar indentado (ver indentação no glossário na Unidade 1).
```

Exemplo:

```
valor = 13
if valor > 10:
   print(“Valor é maior que 10!”)

#Saída após a execução:
Valor é maior que 10!
```

A primitiva if pode ser acompanhada de um else (senão), que não é obrigatório. O else é usado para definir um bloco de código que será executado se a condição do if for falsa e a sintaxe é a seguinte:

```
if condição:
    # Bloco de código a ser executado se condição TRUE
else:
    # Bloco de código a ser executado se condição FALSE
```

Exemplo:

```
valor = 13
if valor > 20:
    print("Valor é maior que 20!")
else:
    print("Valor não é maior que 20!")
# como valor > 20 retorna FALSE, a mensagem "valor não é maior que 20" será impressa.


#Saída após a execução:
Valor não é maior que 20!
```

A primitiva if também pode ser acompanhada de elif (abreviação de “else if”, senão se) que não é obrigatória, pode ser repetida mais de uma vez dentro da estrutura e usada para testar múltiplas condições em sequência. Se a condição do if retornar FALSE a condição do próximo elif será verificada. Se essa também retornar FALSE, a execução passa para o próximo elif ou else.

```
if condição1:

    # Bloco de código se condição1 retorna TRUE

elif condição2:

    # Bloco de código se condição2 retorna TRUE

else:

    # Bloco de código se as condições
anteriores forem FALSE


Exemplo:

valor = 7

if valor > 10:

    print(“Valor é maior que 10!”)

elif valor > 5:

    print(“Valor é maior que 5, mas menor ou igual a 10!”)

else:

    print(“Valor é menor ou igual a 5!”)

# como `valor > 10` retorna FALSE, mas valor > 5
retorna TRUE, a mensagem “valor é maior que 5,
mas menor ou igual a 10” será impressa. Execute
esse exemplo substituindo o valor por 12 e 5


#Saída após a execução:

Valor é maior que 5, mas menor ou igual a 10!
```

Uma possibilidade utilizada é chamada de if Aninhado, que consiste em aninhar estruturas if dentro de outras estruturas if para criar condições mais complexas, onde uma condição depende do resultado de outra. Importante salientar que pode tornar o código mais difícil de entender, especialmente se houver muitos níveis de aninhamento.

```
if condição1:
     if condição2:
            #Bloco de código se ambas as condições forem TRUE
     else:
          # Bloco se condição1 TRUE e condição2 FALSE
else:
     # Bloco de código se condição1 FALSE
```

```
Exemplo:
valor1 = 10
valor2 = 20
if valor1 > 5:
    if valor2 > 15:
        print(“Valor1 > 5 e valor2 > 15”)
    else:
        print(“Valor1 > 5, mas valor2 não é > 15”)
else:
    print(“Valor1 não é > 5”)

# como `valor1 > 5` e `valor2 > 15` são ambos TRUE,
a mensagem “valor1 > 5 e valor2 > 15” será impressa.
Execute e teste com outros valores para valor1 e valor2.
#Saída após a execução:
Valor1 > 5 e valor2 > 15
```

Exemplo2

```
#fonte: https://www.ic.unicamp.br/~wainer/cursos/1s2024/102/aula03.html
print("Digite q, r, ou c (quadrado, retângulo ou círculo):")
a = input()
if a == "q":
    x = input("Digite o tamanho do lado do quadrado: ")
    l = float(x)
    print("A área é :", l*l)
elif a == "r":
    x1 = input("Digite o tamanho de um lado do retângulo: ")
    l1 = float(x1)
    x2 = input("Qual o tamanho do outro lado do retângulo: ")
    l2 = float(x2)
    print("A área é :", l1*l2)
elif a == "c":
    x = input("Digite o tamanho do raio: ")
    r = float(x)
    print("A área é :", 3.1415*r*r)
else:
    print("Opção inválida!")
```

3.1.1 Considerações Sobre Operadores Lógicos em Condições

É possível usar os operadores lógicos `and`, `or` e `not` para combinar múltiplas condições, eles são fundamentais na construção de condições mais complexas dentro de estruturas condicionais. O and retorna TRUE se todas as condições forem verdadeiras. O or retorna TRUE se pelo menos uma das condições for verdadeira. Já o not inverte o valor da condição. A seguir exemplos do uso de and, or e not:

```
exemplo and e or:

idade = int(input('Qual sua idade: ') )

if idade >= 18 and idade <= 70:

     print('Você é obrigado a votar!')

elif idade >= 16 or idade >= 70:

     print('Você já pode votar, mas não é obrigado!')

else:

     print('Você não pode votar')


exemplo or:

idade = 65

ehEstudante = True

if idade >= 65 or ehEstudante:

     print("Você paga meio ingresso.")

else:

     print("Você não paga meio ingresso.")

# para pagar meio ingresso basta uma condição ser TRUE


exemplo not:

idade1 = 12

idade2 = 13

varLogica = idade1 == idade2
```

```
if not varLogica:
     print("Idades diferentes")
else:
     print("As idades são iguais!")
```

3.1.2 Curiosidades Sobre a Condicional if

Uma forma de simplificar o uso da condicional if é através do Operador Ternário, no Python é conhecido como Expressão Condicional. O operador ternário permite escrever uma condição `if-else` em uma única linha.

```
<expressao1> if <condicao> else <expressao2>
```

Primeiro a condição é avaliada (ao invés de expressao1), se a condição for verdadeira, expressao1 é avaliada e seu valor é retornado; caso contrário, expressao2 é avaliada e seu valor retornado. Segue um exemplo:

```
valor = int(input("Digite um valor inteiro: "))
print ("Valor par" if valor % 2 == 0 else "Valor impar")
#Saída após a execução digitando 5:
Digite um valor inteiro: 5
Valor impar
exemplo 2:
valor = 10
mensagem = "valor > 5" if valor > 5 else "valor <= 5"
print(mensagem)
#como valor > 5 é TRUE, `mensagem` receberá "valor > 5".

#Saída após execução
valor > 5
```

- A indentação em Python é crucial para definir os blocos de código associados ao if, elif, e else (ver no glossário da unidade 1).
- Usar if, elif, e else de forma clara e organizada para manter o código legível.
- Se uma condição for muito complexa, considere dividi-la em várias condições ou usar funções para melhorar a legibilidade do código.
- O if é uma das estruturas mais fundamentais em Python e é amplamente utilizado para controlar o fluxo de execução do código com base em condições.

3.2 Laço While

Muitas aplicações cotidianas só podem ser executadas com a implementação de repetição de código, seja com um número conhecido de repetições ou com um número não conhecido de repetições durante a fase de desenvolvimento do programa.

Por exemplo, para desenvolver um script que calcule a média de idade de uma turma de 36 alunos, o número de repetições é conhecido, a estrutura de repetição é importante para que não seja necessário repetir o texto no código fonte 36 vezes. Isso seria possível, mas na medida que o número de entradas aumente pode se tornar inviável repetir no código fonte. Neste exemplo imagine que o código tenha input solicitando a idade de cada aluno. A estrutura de laço que será apresentada nesta seção evita a repetição de 36 input(“Digite a idade do aluno: “) no código fonte.

É possível apresentar o caso onde o número de repetições não é conhecido antes da execução do código, para esse mesmo exemplo do parágrafo anterior, adicionando a regra para cálculo da média somente para os alunos presentes na aula, já torna desconhecido o número de repetições.

A primitiva while em Python é uma estrutura de controle de fluxo que permite a execução repetida de um bloco de código enquanto uma condição específica for verdadeira (TRUE). A avaliação da expressão é realizada da mesma forma que no comando if. Em função da repetição das instruções do bloco do while enquanto a condição for verdadeira, essa estrutura é chamada de laço. A forma mais simples do while é:

```
while condição:
    # Bloco a ser executado enquanto a condição TRUE
# A condição é uma expressão que pode ser avaliada como `True` (verdadeiro) ou `False` (falso) como já apresentado no if.
# O Bloco de código é o código que será executado repetidamente enquanto a condição for verdadeira. Deve estar indentado (ver indentação no glossário da Unidade 1).
```

Exemplo:

```
contador = 0
while contador < 5:
    print("Contador:", contador)
    contador += 1
# Esse bloco de código dentro do while será executado enquanto `contador < 5` for TRUE. A cada iteração, o valor de `contador` é incrementado em 1. O laço termina quando o `contador` atinge 5.
#Saída após a execução:
Contador: 0
Contador: 1
Contador: 2
Contador: 3
Contador: 4
```

Exemplo 2 (média de idade de uma turma de 4 alunos):

```
somaIdade = 0
cont = 1
while cont <= 4:
     idade = int(input(f"Digite a idade do aluno {cont}: "))
     somaIdade += idade
     cont += 1
print(f"A média de idade é {somaIdade/4} anos")

#Saída após a execução digitando 20, 23, 22 e 25:
Digite a idade do aluno 1: 20
Digite a idade do aluno 2: 23
Digite a idade do aluno 3: 22
Digite a idade do aluno 4: 25
A média de idade é 22.5 anos
```

Uma situação que pode acontecer, propositalmente ou não, é conhecida como Laço Infinito (ou loop infinito) , se a condição do while nunca se tornar FALSE, o laço continuará indefinidamente, criando um laço infinito. Por exemplo:

```
while True:
     print("Este é um laço infinito!")
# como a condição TRUE nunca muda, o laço continuará executando
indefinidamente. Para interromper um laço infinito, você pode
usar "<Ctrl> + C" no terminal ou usar uma instrução break
dentro do laço.
```

A primitiva break interrompe imediatamente a execução do laço, mesmo que a condição do while ainda seja TRUE. Além do break, também é utilizada para manipulação de laços a primitiva continue, a qual pula o restante do bloco de código dentro do laço e volta a verificar a condição do while. A seguir seguem exemplos com a utilização do break e do continue:

Exemplo Break:

```
while True:
    entrada = input("Digite algo (feito interrompe): ")
    if entrada == "feito":
        break
    print(entrada)
print("Feito!")
# o laço será interrompido quando a variável entrada receber a palavra "feito", toda em minúsculo.
```

Exemplo com continue:

```
contador = 0
while contador < 10:
    contador += 1
    if contador % 3 == 0:
        continue
    print("Contador:", contador)
# Quando a variável contador for múltipla de 3, o `continue` será executado, pulando a instrução `print` para essa iteração. Como resultado, não serão impressos os múltiplos de 3 no laço de 0 até 10.
#Saída após a execução:
Contador: 1
Contador: 2
Contador: 4
Contador: 5
Contador: 7
Contador: 8
Contador: 10
```

Uma característica que o Python apresenta em relação ao laço while é a possibilidade do uso da primitiva else. O else em um laço while é executado quando a condição do while se torna FALSE. No entanto, se o laço for interrompido por um break, o bloco else não será executado. Segue a sintaxe:

```
while condição:
    # Bloco de código a ser executado enquanto condição TRUE
else:
    # Bloco a ser executado quando condição se tornar FALSE
```

Exemplo:

```
contador = 0
while contador < 5:
    print("Contador:", contador)
    contador += 1
else:
    print("Laço terminado!")
# Neste exemplo, o bloco else será executado após o laço while terminar normalmente (quando contador atingir 5).
```

Outra possibilidade para utilização da estrutura é o aninhamento de laço while dentro de outros laços while para criar estruturas de repetição mais complexas. Segue a sintaxe:

```
while condição1:
    # Bloco a ser executado enquanto condição1 TRUE
    while condição2:
        # Bloco ser executado enquanto condição2 TRUE
```

```
Exemplo percorrendo uma tabela 3 x 3:
linha = 1
while linha <= 3:
     coluna = 1
     while coluna <= 3:
          print(f”linha: {linha}, coluna: {coluna}”)
          coluna += 1
     linha += 1
#Saída após a execução:
linha: 1, coluna: 1
linha: 1, coluna: 2
linha: 1, coluna: 3
linha: 2, coluna: 1
linha: 2, coluna: 2
linha: 2, coluna: 3
linha: 3, coluna: 1
linha: 3, coluna: 2
linha: 3, coluna: 3
# Neste exemplo, o laço externo percorre as linhas da matriz
e o laço interno percorre as colunas, resultando em uma saída
que percorre em cada linha 3 colunas.
```

Uma importante aplicação para o laço while na linguagem Python é para a validação de entradas. O while é frequentemente usado para validar entradas do usuário, garantindo que o programa continue solicitando uma entrada até que uma condição seja satisfeita. A seguir um exemplo dessa aplicação:

```
while True:
    idade = input("Digite sua idade: ")
    if idade.isdigit() and int(idade) >= 0:
        print("Idade válida:", idade)
        break
    else:
        print("Idade inválida. Tente novamente.")
# o laço continuará solicitando a idade do usuário até que uma idade válida (um número não negativo) seja fornecida.
# Cabe uma observação sobre o idade.isdigit(), isdigit() é um método que que verifica se todos os caracteres em uma string são dígitos, ou seja dados que devem conter apenas valores numéricos mesmo em formato de string.
```

Algumas observações que devem ser consideradas sobre os laços e que podem gerar dúvidas. Variáveis contadoras realizam a contagem de ocorrências de um determinado valor (ou situação) e normalmente são utilizadas como critério de parada para o laço. As variáveis acumuladoras são responsáveis por armazenar os resultados de somatórios e produtórios de valores dentro dos laços. Outra consideração na implementação dos laços é certificar-se de que a condição de parada do while se torne FALSE em algum momento para evitar laços infinitos, a menos que seja intencionalmente um laço infinito. O while é uma estrutura de controle ideal para situações onde você precisa repetir um bloco de código enquanto uma condição específica for verdadeira. Com o uso adequado de break, continue e else, é possível controlar o fluxo do laço de maneira eficiente e clara.

3.3 Laço for

Outra alternativa para implementar laços de repetição é o for, presente na maioria das linguagens de programação ele é muito utilizado em Python para percorrer sequências previamente conhecidas como alternativa ao while. Sequências são coleções ordenadas embutidas, que podem ser strings (conjunto de caracteres), listas (são coleções de itens heterogêneos), dicionários (coleção que guarda valores multidimensionais para cada índice) e tuplas (são sequências imutáveis). Essas estruturas serão apresentadas na unidade 4, além disso algumas facilidades da utilização do for para manipular essas estruturas. Sintaxe do laço for:

```
for <variável> in <objeto iterável>:
     bloco de instrução
```

Exemplo simples:

```
for item in [3,4,5,6]:
     print(item)
#Para cada elemento da lista[3,4,5,6] execute o bloco de código a seguir e a cada execução, atribua à var item um item da lista
#Saída após a execução:
3
4
5
6
```

exemplo 2:

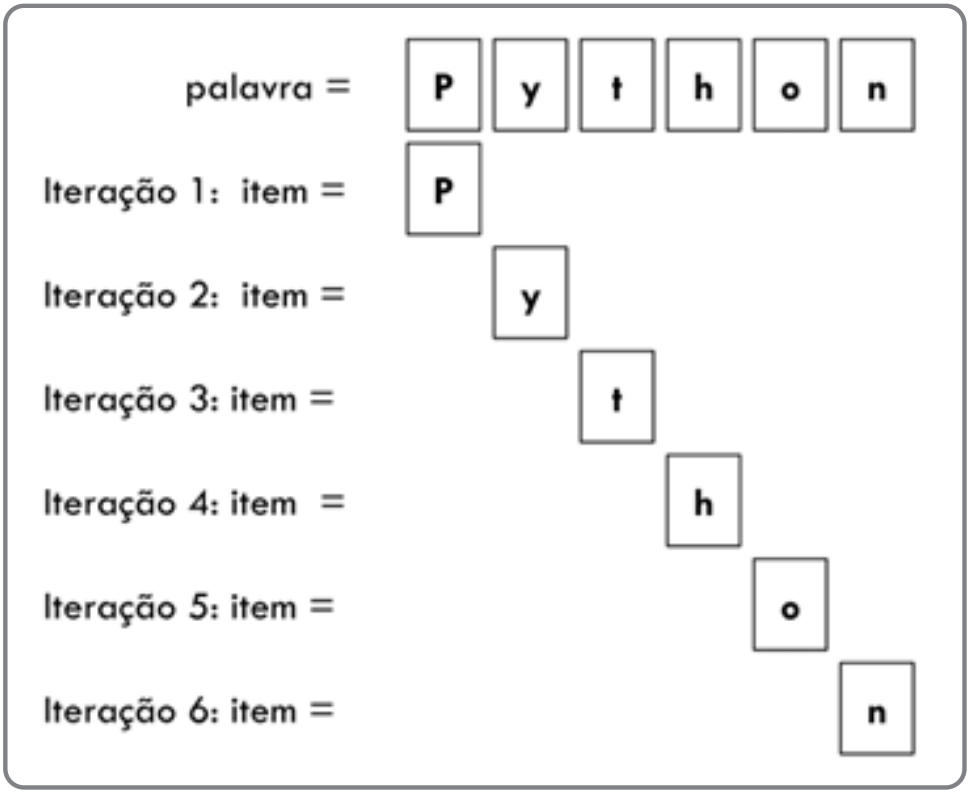
```
palavra = 'Python'
for item in palavra:
     print(item)
```

```
# item e palavra são nomes de variável. A instrução do laço
for atribuirá repetidamente os caracteres da string palavra
à variável item.


#Saída após a execução:
P
y
t
h
o
n
```

Cada volta do laço é chamada de iteração (sem n), neste exemplo anterior serão executadas 6 iterações, uma para cada caracter. A figura a seguir apresenta a representação gráfica do comportamento da execução deste exemplo:

Figura - Representação gráfica da execução de um laço.



Fonte: autoria própria.

A função range() é uma importante função utilizada para implementação de laços for. Como o próprio nome já adianta, ela permite a criação de intervalos numéricos. Esse intervalo é utilizado para controlar as iterações do laço. Sua sintaxe pode ter as seguintes três formas, sendo que seu único parâmetro obrigatório é o fim.

```
range(fim) # intervalo de 0 (zero) até fim - 1

range(inicio, fim) # intervalo de inicio até fim - 1

range(inicio, fim, passo) # intervalo de inicio até fim -1, aumentando ou diminuindo de passo em passo. Quando o parâmetro passo é omitido, o padrão é 1 o passo.

#É importante ressaltar que na versão 3.x do Python, a função range() retorna um objeto iterável e não mais uma lista com elementos (Iteráveis são objetos capazes de retornar seus membros um de cada vez - eles podem ser iterados. As estruturas de dados incorporadas populares do Python, como listas, tuplas e conjuntos, são qualificadas como iteráveis). Por essa razão devemos converter o retorno para listas com a função list().
```

A seguir seguem alguns exemplos que apresentam intervalos gerados com a função range(), cabe salientar que o list() não é necessário quando é utilizada a função range() no laço for, conforme exemplos na sequência:

```
# Gerar lista com (fim)

print(list(range(8)))

#Saída após a execução:

[0, 1, 2, 3, 4, 5, 6, 7]


# Gerar com (inicio, fim)

print(list(range(2, 7)))

#Saída após a execução:

[2, 3, 4, 5, 6]
```

```
# Gerar com (inicio, fim, passo)
print(list(range(2, 9, 3)))
#Saída após a execução:
[2, 5, 8]

# Exemplo intervalo decrescente
print(list(range(0, -10, -1)))
#Saída após a execução:
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

A seguir exemplos da utilização da função range() associada ao for, onde é importante salientar que a cada iteração, a variável associada ao for receberá um valor do intervalo. Nos 3 exemplos seguintes essa variável foi chamada de i.

```
# Exemplo intervalo crescente
for i in range(5):
     print(i)
#Saída após a execução:
0
1
2
3
4

# Exemplo intervalo decrescente
for i in range(0,-5,-1):
     print(i)
#Saída após a execução:
```

```
0
-1
-2
-3
-4
```

```
# Exemplo intervalo com passo 2
for i in range(0,10,2):
    print(i)
```

Resultado da execução:

```
0
2
4
6
8
```

Assim como a implementação do laço while, o for também pode ser acompanhado pelo else, para confirmar o final do laço. A seguir é apresentado um exemplo desta estrutura.

```
for i in range(5):
    print(i)
else:
    print('após iteração final')
Resultado da execução:
0
1
```

```
2

3

4

após iteração final
```

3.4 Vetores e Matrizes

As matrizes são estruturas fundamentais da computação científica, representando dados de forma unidimensional, bidimensional ou multidimensional. No contexto da programação as matrizes são chamadas de *arrays.* Trabalhar com matrizes numéricas é essencial para diversas áreas, incluindo ciência de dados, aprendizado de máquina, processamento de imagens e simulações científicas.

Uma matriz é uma coleção de variáveis do mesmo tipo e esta é referenciada por um nome comum. São estruturas que servem para guardar dados, e organizá-los. Seu objetivo é ser um espaço na memória do computador que armazena elementos. Cada elemento de uma matriz é acessado através de um índice. O índice 0 (zero) corresponde ao primeiro elemento de uma matriz, e o índice mais alto, representa o último elemento desta matriz. Basicamente são trabalhadas três classificações de matrizes quanto ao número de dimensões:

- **Matrizes Unidimensionais:** São matrizes de apenas uma dimensão, e são geralmente chamadas de vetores ou listas (estáticas). Exemplo pode ser trabalhado como as 4 notas de uma prova.

0	1	2	3
7.5	8.6	8.9	9.5

- **Matrizes Bidimensionais:** São matrizes de duas dimensões, um exemplo prático podem ser trabalhados em imagens digitais, criadas como matrizes de pixels.

	0	1	2	3
0	255	155	60	0
1	240	100	55	0

- **Matrizes Multidimensionais:** São matrizes de n dimensões. Um exemplo prático pode ser trabalhado em vídeos digitais, criados como sequências de imagens. Abaixo um exemplo de uma matriz 3x2x4, ou seja, profundidade 3, duas linhas e quatro colunas.



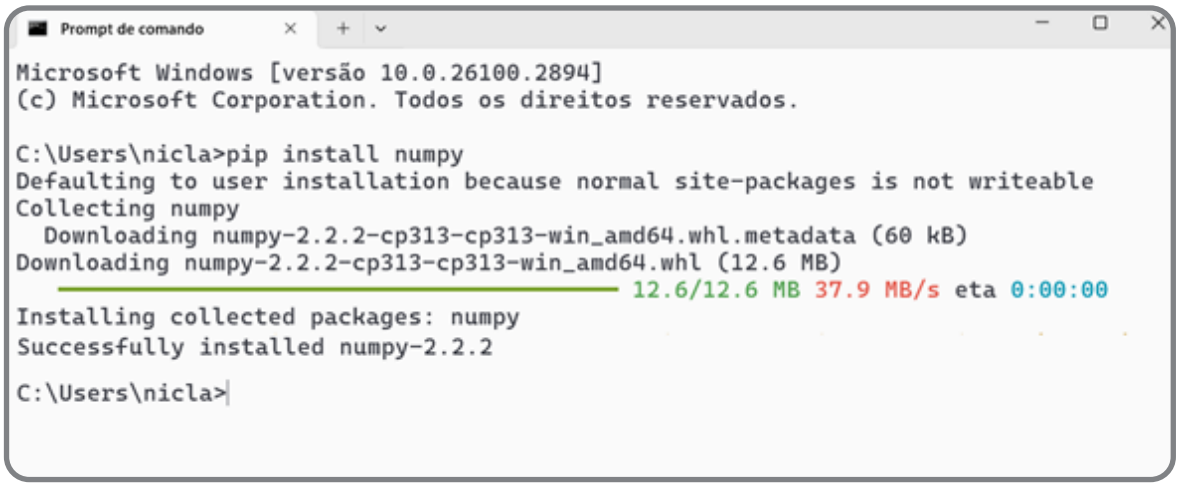
Python é uma ferramenta poderosa para trabalhar com dados em função de sua capacidade de manipular matrizes de forma eficiente. O domínio dessas operações é crucial para a análise e interpretação de informações em diferentes contextos. Aprofundar-se no uso de matrizes em Python é um passo importante para quem deseja avançar em áreas técnicas e científicas.

Em Python é possível representar matrizes usando listas ou listas de listas (listas serão apresentadas na Unidade 4), mas essa abordagem não é tão eficiente quanto o uso de arrays da NumPy (ver o conceito de biblioteca no glossário da Unidade 1), especialmente para operações matemáticas complexas. Numpy se destaca quando há grandes quantidades de dados do mesmo tipo para serem processados pela CPU. Seu nome vem de *Numerical Python* e essa é a biblioteca mais utilizada para manipulação de matrizes em Python. Para explorar mais, consulte a documentação oficial da NumPy em https://numpy.org/doc/, que oferece um guia completo sobre todas as funcionalidades disponíveis.

A maioria dos arrays NumPy tem algumas restrições. Todos os elementos da matriz devem ser do mesmo tipo de dados. Uma vez criado, o tamanho total do array não pode ser alterado. A forma deve ser “retangular”, não “irregular”; por exemplo, cada linha de uma matriz bidimensional deve ter o mesmo número de colunas.

A biblioteca NumPy fornece uma gama de funções matemáticas que facilitam operações como adição, subtração, multiplicação e transposição de matrizes. Para começar a trabalhar com NumPy, o primeiro passo é instalar a biblioteca, que pode ser feita através do comando <pip install numpy> digitado num terminal, o resultado no sistema operacional Windows, em um terminal Cmd é apresentado na Figura a seguir:

Figura - Instalação da biblioteca Numpy



Fonte: Autoria própria

Após instalar a NumPy, ela pode ser importado para o código Python utilizando a seguinte linha no cabeçalho do script:

```
import numpy as np
```

```
# Essa é a convenção normalmente encontrada e permite o acesso aos recursos do NumPy com um prefixo ( np.), ao mesmo tempo em que distingue os recursos da NumPy de outras bibliotecas.
```

Assim como as sequências Python, as matrizes NumPy são “indexadas em 0”: o primeiro elemento da matriz é acessado usando index 0, não 1. É uma prática comum se referir a elementos de uma matriz pelo índice da linha primeiro e pelo índice da coluna em segundo. A seguir é apresentado graficamente uma matriz e na sequência um script para criar a mesma matriz 2x3 inicializada utilizando uma sequência Python como uma lista.

	0	1	2
0	1	2	3
1	4	5	6

```
import numpy as np

matriz = np.array([[1, 2, 3],[4, 5, 6]])

print(f”Matriz:\n{matriz}”)  # /n quebra linha
```

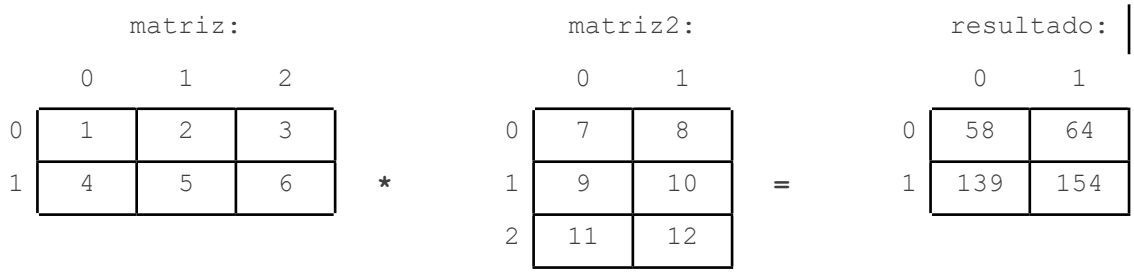
```
# Acessando elementos
print(f”Dado na 1a linha e 2a coluna:\n{matriz[0, 1]}”)

#Saída após a execução:
Matriz:
[[1 2 3]
 [4 5 6]]
Dado na 1a linha e 2a coluna:
2

#Outras operações com matrizes adicionando ao script anterior:
matriz _ soma = matriz + 10  # Soma 10 a cada elemento
print(f”Matriz após somar 10:\n{matriz _ soma}”)
#Saída após a execução:
Matriz após somar 10:
[[11 12 13]
 [14 15 16]]

# Multiplicação de matrizes
matriz2 = np.array([[7, 8],[9, 10],[11, 12]])
resultado = np.dot(matriz, matriz2)
print(f”Resultado da multiplicação:\n{resultado}”)
#Saída após a execução:
Resultado da multiplicação:
[[ 58  64]
 [139 154]]
```

Representação gráfica deste último exemplo:



Principais operações com matrizes em funções disponibilizadas pela Numpy podem ser estudadas na documentação oficial em numpy.org. Para facilitar a criação de matrizes, por exemplo existem algumas funções básicas dentre as quais `np.array()`, `np.zeros()`, `np.ones()`, `np.eye()` (matriz identidade). É importante exercitar, testar e verificar os resultados.

Síntese da Unidade

Aqui está um resumo dos principais temas abordados nas 5 seções desta Unidade, sobre condicionais, laços e matrizes.

1. If, elif e else

- Apresenta as bases de primitivas condicionais.
- Apresenta a primitiva if e suas variantes.

2. Laço While

- Introduz a temática de laços em programação.
- Apresenta o tipo de laço while.

3. Laço For

- Apresenta o tipo de laço for.
- Apresenta a função range().

4. Vetores e Matrizes

- Apresenta introdução aos arrays.
- Apresenta a biblioteca Numpy..

Esse resumo abrange os temas principais e fornece uma visão geral do conteúdo abordado na unidade

Bibliografia

BANIN, S. L. Python 3 - **Conceitos e Aplicações - Uma abordagem didática.** Rio de Janeiro: Érica, 2018. E-book. p.Capa. ISBN 9788536530253. Disponível em: minhabiblioteca.com.br/reader/books/9788536530253/. Acesso em: 15 out 2024.

DEEPSEEK. Disponível em: https://deepseek.com/. Acesso em: 25 fev. 2025.

LAMBERT, K. A. **Fundamentos de Python:** primeiros programas. São Paulo: Cengage Learning Brasil, 2022. E-book. p.i. ISBN 9786555584301. Disponível em: minhabiblioteca.com.br/reader/books/9786555584301/. Acesso em: 15 out 2024.

NUMPY. Disponível em: https://numpy.org/. Acesso em: 25 fev. 2025.

SEVERANCE, C. R. **Python para Todos.** CreateSpace Independent Publishing Platform. 2016. Disponível em: https://do1.dr-chuck.com/pythonlearn/PT_br/pythonlearn.pdf. Acesso em: 15 out 2024.

RAMALHO, L. **Aprenda a Programar. Creative Commons**. Disponível em: https://wiki.python.org.br/AprendaProgramar ou em . Acesso em: 14 fev 2025.

POE. Disponível em: https://poe.com/. Acesso em: 25 fev. 2025.

PYTHON. **Documentação.** Disponível em: https://www.python.org/. Acesso em: 25 fev. 2025.

WAINER, J. **Expressões Relacionais, Lógicas, Comandos Condicionais e Funções**. Disponível em: https://www.ic.unicamp.br/~wainer/cursos/1s2024/102/aula03.html. Acesso em: 25 fev. 2025.

Programação Modular

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Baseado na lógica da Programação Estruturada, apresentar a programação modular, além de questões avançadas.

Objetivos Específicos

- Introduzir a programação modular.
- Estudar estruturas avançadas como tuplas, listas e dicionários.
- Introdução ao trabalho com arquivos.
- Desenvolver exercícios de fixação.

Questões Contextuais

- Qual a importância do desenvolvimento modular de software?
- Como trabalhar com tuplas, listas e dicionários?
- Qual a importância de manipulação de arquivos?

Editora LaSalle

4.1 Questões Fundamentais Sobre Programação Modular

Todos os scripts desenvolvidos no decorrer deste material são programas pequenos que não passam de 50 linhas de código, talvez os exercícios de fixação propostos nas unidades superem esse número, mas com certeza não chegam perto do tamanho de uma aplicação profissional. No mundo real, softwares modestos têm milhares de linhas de código, e essa contagem chega aos milhões quando se fala de grandes aplicativos como o Facebook, estimado em 20 milhões de linhas e o Windows 50 milhões. Mas nada se aproxima ao somatório da maioria dos serviços Google, que segundo estimativas chega aos 2 bilhões de linhas de código, isso inclui todos os serviços, como o Google Maps, Plus, Pay, Photos, Gmail e Drive, mas não conta os sistemas operacionais Android e Chrome OS. Esses números podem surpreender, porém não são corretos porque foram apresentados em 2015, por uma gerente de engenharia da Google (https://www.wired.com/2015/09/google-2-billion-lines-codeand-one-place/).

Independente do tamanho gigante do exemplo anterior ou pequeno dos exemplos deste material para aprendiz, é importante ter em mente que na medida que se entra no mundo profissional do desenvolvimento de software as linhas de código vão se multiplicando. A boa notícia é que em projetos muito grandes não é somente um programador que trabalha no código, no mundo profissional há forte tendência ao trabalho em equipes. Para que todos se entendam, são criadas diversas regras, convenções e padrões. Neste sentido, uma boa dica de leitura é o livro Código Limpo. Uma das primeiras técnicas que ajuda os programadores a se entenderem é a programação modular.

> Nos primórdios da programação, formávamos nossos sistemas com rotinas e sub-rotinas. Já na era do Fortran e do PL/1, usávamos programas, subprogramas e funções. De tudo isso, apenas a função prevaleceu. As funções são a primeira linha de organização em qualquer programa. (MARTIN ,2009)

Quando se trabalha com scripts maiores é fundamental a divisão em módulos ao invés de criar um bloco de código corrido e monolítico onde a aplicação inteira está expressa em uma única sequência de comandos. Por isso, o termo programação modular é utilizado. Na programação estruturada, o termo programação modular é aplicado porque o código é dividido em subprogramas que encapsulam blocos de código relacionados, esses blocos ou módulos também são chamados de funções. Funções ajudam a dividir o problema em partes menores e mais manejáveis. Isso melhora a

organização e a reutilização do código. No decorrer do aprendizado alguns outros conceitos associados a programação modular surgiram, dentre eles, objetos, classes, módulos e pacotes.

A programação modular oferece vários benefícios para os programadores e equipes de programadores, como a facilidade de manutenção. Módulos bem definidos tornam mais fácil identificar, corrigir e atualizar partes específicas do código sem afetar o sistema como um todo. Reutilização de código na medida em que os módulos podem ser reutilizados em diferentes projetos ou partes do mesmo projeto, economizando tempo e esforço. Colaboração em equipes de desenvolvimento, diferentes programadores podem trabalhar em módulos separados simultaneamente, facilitando a colaboração. Melhoria da legibilidade, pois código organizado em módulos é geralmente mais fácil de entender, pois cada módulo tem uma função clara. Facilidade de Testes porque os módulos podem ser testados individualmente antes de serem integrados, o que ajuda a garantir que cada parte do sistema funcione corretamente.

Na realidade, desde os primeiros exemplos o leitor já está utilizando funções desde o início dos exemplos práticos deste material, quando executou um exemplo utilizando print() ou input() por exemplo. Esses são dois exemplos de funções embutidas (built-in), também chamadas de funções internas, disponibilizadas por Python e pela maioria das linguagens de programação. Python possui uma vasta gama destas funções que estão sempre disponíveis para uso, sem a necessidade de importar módulos adicionais. Essas funções são essenciais para realizar tarefas comuns em programação. Abaixo uma listagem com uma série de funções embutidas que podem ser utilizadas. Essas são apenas algumas das muitas funções *built-in* disponíveis em Python. Elas ajudam a realizar tarefas comuns de forma eficiente. Para mais detalhes, é possível consultar https://docs.python.org/3/library/functions.html .

Funções de Conversão e Tipo

1. int(): Converte um valor para inteiro.

```
x = int("10")  # x será 10
```

2. float(): Converte um valor para float (número de ponto flutuante).

```
x = float("10.5")  # x será 10.5
```

3. str(): Converte um valor para string.

```
x = str(10)  # x será "10"
```

4. bool(): Converte um valor para booleano (`True` ou `False`).

```
x = bool(1)  # x será True
```

5. list(): Converte um iterável (como uma tupla ou string) em uma lista.

```
x = list("hello")  # x será ['h', 'e', 'l', 'l', 'o']
```

6. tuple(): Converte um iterável em uma tupla.

```
x = tuple([1, 2, 3])  # x será (1, 2, 3)
```

7. set(): Converte um iterável em um conjunto (remove duplicatas).

```
x = set([1, 2, 2, 3])  # x será {1, 2, 3}
```

8. dict(): Cria um dicionário.

```
x = dict(a=1, b=2)  # x será {'a': 1, 'b': 2}
```

Funções Matemáticas

1. abs(): Retorna o valor absoluto de um número.

```
x = abs(-10)  # x será 10
```

2. round(): Arredonda um número para o inteiro mais próximo ou para o número de casas decimais especificado.

```
x = round(10.567, 2)  # x será 10.57
```

3. min(): Retorna o menor valor em um iterável.

```
x = min([1, 2, 3])  # x será 1
```

4. max(): Retorna o maior valor em um iterável.

```
x = max([1, 2, 3])  # x será 3
```

5. sum(): Retorna a soma de todos os elementos em um iterável.

```
x = sum([1, 2, 3])  # x será 6
```

Funções de Iteração

1. range(): Gera uma sequência de números.

```
x = list(range(5))  # x será [0, 1, 2, 3, 4]
```

2. enumerate(): Retorna um objeto enumerado (índice e valor) de um iterável.

```
for i, value in enumerate(['a', 'b', 'c']):
    print(i, value)  # Saída: 0 a, 1 b, 2 c
```

3. zip(): Combina dois ou mais iteráveis em pares.

```
x = list(zip([1, 2, 3], ['a', 'b', 'c']))  # x será [(1, 'a'), (2, 'b'), (3, 'c')]
```

Funções de Entrada e Saída

1. print(): Exibe valores na saída padrão (normalmente o console).

```
print("Hello, World!")  # Saída: Hello, World!
```

2. input(): Lê uma string da entrada padrão.

```
name = input("Digite seu nome: ")
```

Funções de Verificação

1. len(): Retorna o comprimento (número de elementos) de um objeto.

```
x = len("hello")  # x será 5
```

2. type(): Retorna o tipo de um objeto.

```
x = type(10)  # x será <class 'int'>
```

3. isinstance(): Verifica se um objeto é uma instância de uma classe ou tipo específico.

```
x = isinstance(10, int)  # x será True
```

Funções de Manipulação de Strings

1. str.upper(): Converte uma string para maiúsculas.

```
x = "hello".upper()  # x será "HELLO"
```

2. str.lower(): Converte uma string para minúsculas.

```
x = "HELLO".lower()  # x será "hello"
```

3. str.strip(): Remove espaços em branco no início e no final de uma string.

```
x = "  hello  ".strip()  # x será "hello"
```

4. str.split(): Divide uma string em uma lista de substrings com base em um delimitador.

```
x = "hello world".split()  # x será ['hello', 'world']
```

Funções de Ordenação

1. sorted(): Retorna uma nova lista ordenada a partir de um iterável.

```
x = sorted([3, 1, 2])  # x será [1, 2, 3]
```

2. reversed(): Retorna um iterável invertido.

```
x = list(reversed([1, 2, 3]))  # x será [3, 2, 1]
```

Funções de Arquivo

1. open(): Abre um arquivo e retorna um objeto de arquivo.

```
file = open("arquivo.txt", "r")
content = file.read()
file.close()
```

Funções de Execução de Código

1. eval(): Avalia uma string como uma expressão Python.

```
x = eval("10 + 5")  # x será 15
```

2. exec(): Executa um código Python dinamicamente.

```
exec("x = 10 + 5")  # x será 15
```

Funções de Memória e Identificação

1. id(): Retorna a identidade (endereço de memória) de um objeto.

```
x = id("hello")  # Retorna um número único que identifica o objeto
```

2. hash(): Retorna o valor hash de um objeto (útil para chaves de dicionários).

```
x = hash("hello")  # Retorna um valor hash único
```

Funções de Coleções

1. all(): Retorna `True` se todos os elementos de um iterável são verdadeiros.

```
x = all([True, 1, "hello"])  # x será True
```

2. any(): Retorna `True` se pelo menos um elemento de um iterável é verdadeiro.

```
x = any([False, 0, "hello"])  # x será True
```

Funções de Mapeamento e Filtragem

1. map(): Aplica uma função a todos os itens de um iterável.

```
x = list(map(lambda a: a * 2, [1, 2, 3]))  # x será [2, 4, 6]
```

2. filter(): Filtra elementos de um iterável com base em uma função.

```
x = list(filter(lambda a: a > 1, [1, 2, 3]))  # x será [2, 3]
```

Funções de Ajuda e Documentação

1. help(): Exibe a documentação de um objeto ou função.

```
help(print)  # Exibe a documentação da função print
```

2. dir(): Retorna uma lista de atributos e métodos de um objeto.

```
x = dir(str)    # Retorna uma lista de métodos disponíveis para strings
```

Em Python, a programação modular vai além do uso de funções embutidas. Ela permite que o programador organize seu código em módulos, pacotes e estruturas mais complexas, promovendo reutilização, manutenção e clareza do código. Uma das práticas mais importantes para os programadores é a criação de funções. Essas práticas serão apresentadas na próxima seção deste texto.

Alguns conceitos importantes para o programador iniciante são o de Variáveis Locais, que são variáveis utilizadas somente nas funções que foram definidas (Escopo Local). Variáveis Globais que são variáveis definidas fora das funções (Escopo Global). Parâmetros de Função (ou argumentos) que são variáveis que recebem os valores iniciais na chamada das funções, aparecem listadas entre parênteses na definição das funções, além disso, os parâmetros representam valores que a função precisa para processar o resultado que a função irá produzir. Muitas vezes podem ser definidos como parâmetros formais ou parâmetros reais. Um parâmetro formal é uma variável local vazia listada no cabeçalho e utilizada dentro da função. Um parâmetro real representa um valor ou endereço utilizado na execução da função. Parâmetros reais substituem os parâmetros formais quando ocorre a chamada da função. Por exemplo, a seguir há um exemplo com

a definição da função cumprimentar, com os parâmetros formais meu_nome e idade. No mesmo exemplo, na execução dessa função surgem os parâmetros reais “Fulano de Tal” e 45 substituindo esses parâmetros formais.

4.2 Formas de uma Função

Além das funções embutidas que já vem com o Python, é possível criar as próprias funções para desenvolver, conter a lógica desejada e promover a reutilização. A sintaxe de uma função é definida por três partes: nome da função, parâmetros e o corpo da função, o qual agrupa uma sequência de linhas que representa a lógica implementada. No código abaixo, temos um exemplo de declaração de função em Python. Uma vez que foi definida uma função, é possível reutilizá-la repetidas vezes ao longo do script ou de outros scripts, conforme será apresentado no decorrer dessa unidade.

```
def nome _ da _ funcao(<PARÂMETROS>):
    #código identado faz parte da função


# A sintaxe para chamar a função é a mesma que para funções embutidas:
nome _ da _ funcao()
```

A palavra reservada def, na primeira linha, indica que isso é uma definição de função. O nome da função na sequência, sendo que as regras para nomes de funções são as mesmas que para nomes de variáveis: letras, números e alguns sinais de pontuação são permitidos, mas o primeiro caractere não pode ser um número. Em seguida, entre parênteses vem os parâmetros que serão recebidos, é possível ter zero, um ou mais, muitas vezes chamados de parâmetros formais. Ainda na mesma linha, observe a utilização dos dois pontos (:), que indicam que o código indentado (ver conceito de indentação na Unidade 1) nas linhas abaixo faz parte da função que está sendo criada. É importante ressaltar que a indentação, para respeitar a sintaxe da linguagem, todas as linhas relativas à função estão avançadas em relação à primeira. Para executar a função, de forma semelhante ao que ocorre em outras linguagens, devemos simplesmente chamar seu nome e passar os parâmetros esperados entre parênteses, conforme o código a seguir.

```
#exemplo
def cumprimentar(meu _ nome,idade):
     print(f”Olá {meu _ nome}, de {idade} anos!”)

#para executar a função cumprimentar:
cumprimentar(“Fulano de Tal”,45)
# resultado após a execução:
Olá Fulano de Tal, de 45 anos!

#exemplo 2
def calcular _ area _ retangulo(largura, altura):
     return largura * altura

#para executar a função:
area = calcular _ area _ retangulo(5, 10)
print(f”Área do retângulo = {area}”)
# resultado após a execução:
Área do retângulo = 50
#exemplo 3 - script para apresentar a seguinte saída:
------------------------------------------------------
                Sistema Desenvolvido
------------------------------------------------------
------------------------------------------------------
                  Cadastro de Alunos
------------------------------------------------------
------------------------------------------------------
```

```
                        Menu Principal
-----------------------------------------------------

#primeira tentativa sem uso de função:
print("-----------------------------------------------------")
print("                   Sistema Desenvolvido")
print("-----------------------------------------------------")
print("-----------------------------------------------------")
print("                    Cadastro de Alunos")
print("-----------------------------------------------------")
print("-----------------------------------------------------")
print("                      Menu Principal")
print("-----------------------------------------------------")

#Segunda tentativa com uso função:
def imprimir():
     print("-"*53)

imprimir()
print("                    Sistema Desenvolvido")
imprimir()
imprimir()
```

```
print("                    Cadastro de Alunos")
imprimir()
imprimir()
print("                      Menu Principal")
imprimir()


#Terceira tentativa com uso função:
def imprimir _ mensagem(txt):
     print("-"*53)
     print(txt)
     print("-"*53)


imprimir _ mensagem("                  Sistema Desenvolvido")
imprimir _ mensagem("                  Cadastro de Alunos")
imprimir _ mensagem("                    Menu Principal")
```

As saídas da primeira, segunda e terceira tentativas produzem o mesmo resultado solicitado, mas é possível observar que no desenvolvimento das soluções foi aprimorada a função e o reaproveitamento de código. Esse é um exemplo simples tem a intenção de apresentar a vantagem do uso de modularidade. Mesmo para esse pequeno código é possível melhorar, a ideia se multiplica em códigos maiores.

Um ponto importante a considerar na criação das funções em Python são as convenções de nomenclatura, iniciativas para manter o código legível e padronizado. Dentre as diretrizes e convenções aplicadas é importante usar nomes que descrevam a função, evitando abreviações que possam ser confusas. Por exemplo, calcular_area() é melhor do que c_area(). Outra convenção está associada ao estilo de escrita do nome utilizando o Snake Case, onde nomes de funções devem ser escritos em letras minúsculas e com palavras separadas por underscores (_). Exemplo obter_dados_usuario(). É aconselhável utilizar verbos no início do nome, iniciando os nomes de funções com um verbo que indique a ação que a função executa. Exemplo salvar_arquivo() ou calcular_

media(). Evitar nomes de funções duplicados, certificando-se de que o nome da função não conflita com nomes de funções incorporadas ou de bibliotecas importadas. Essas convenções não são obrigatórias, por isso é importante o programador tomar cuidado para manter um padrão que possa ser reconhecido pela equipe de programadores.

Os exemplos anteriores apresentavam funções que executam suas operações e geram uma saída através de prints na tela, porém essa não é a única maneira da função interagir com a aplicação. Outra forma importante é a função retornar algum valor. A seguir alguns exemplos, considerando que a palavra reservada return é utilizada para retornar o valor e finalizar a função.

```
# Exemplo somar _ numeros, que recebe 2 números e retorna a soma
def somar _ numeros(a, b):
    return a + b


resultado = somar _ numeros(231, 43)
print(f"Resultado da soma = {resultado}")
# Saída após a execução:
Resultado da soma = 274


#Exemplo2 calcular _ quadrado, retorna o quadrado de numero
def calcular _ quadrado(numero):
    return numero ** 2


valor = int(input("Entre com um valor: "))
resultado = calcular _ quadrado(valor)
print(f"Resultado de {valor} ao quadrado = {resultado}")
# Saída após a execução:
Entre com um valor: 9
Resultado de 9 ao quadrado = 81
```

#exemplo3, função que retorna uma lista de números de 1 a 10. (para entender melhor esse exemplo, na próxima seção deste texto será apresentado o uso de listas)

```
def retornar _ numeros():
     numeros = []
     for i in range(1, 11):
          numeros.append(i) #adiciona ao final da lista
     return numeros

resultado = retornar _ numeros()
print(f”Resultado = {resultado}”)
# Saída após a execução:
Resultado = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

#**exemplo4** função que retorna mais de um valor,

```
# neste exemplo o quadrado e o cubo de um número.
def calcular _ quadrado _ e _ cubo(numero):
    quadrado = numero ** 2
    cubo = numero ** 3
    return quadrado, cubo

result = calcular _ quadrado _ e _ cubo(7)
print(f”Quadrado = {result[0]} e cubo = {result[1]}”)
result = calcular _ quadrado _ e _ cubo(3)
```

```
print(f"Quadrado = {result[0]} e cubo = {result[1]}")
# Saída após a execução (executada função 2 vezes no exemplo):
7 - Quadrado = 49 e cubo = 343
3 - Quadrado = 9 e cubo = 27
```

#exemplo5 Função para verificar par ou impar e retorna um def verificar _ par(numero):

```
    return numero % 2 == 0 #verifica o resto da divisão por 2

resultado = verificar _ par(4)
print(f"4 - {resultado}")
resultado = verificar _ par(3)
print(f"3 - {resultado}")
# Saída após a execução (executada função 2 vezes no exemplo):
4 - True
3 - False
```

#exemplo6 Função que faz o cálculo do salário e retorna o valor a ser pago conforme o número de horas trabalhadas. Fonte: https://www.devmedia.com.br/funcoes-em-python/37340

```
def calcular _ pagamento(qtd _ horas, valor _ hora):
    if qtd _ horas <= 40:
        salario = qtd _ horas * valor _ hora
    else:
        h _ excd = qtd _ horas - 40
        salario = 40*valor _ hora+(h _ excd*(1.5*valor _ hora))
    return salario
```

```
qtd _ horas = float(input('Digite quantidade de horas: '))
valor _ hora = float(input('Digite o valor da hora: '))
total _ salario = calcular _ pagamento(qtd _ horas, valor _ hora)
print(f"O valor de seus rendimentos é R${total _ salario:.2f}")
# Saída após a execução
Digite quantidade de horas: 40
Digite o valor da hora: 100
O valor de seus rendimentos é R$4000.00
```

Importante testar os exemplos anteriores, mais de uma vez, modificando os valores de entrada e mesmo o código dos exemplos para uma fixação profunda dos conceitos apresentados.

4.3 Tuplas, Listas e Dicionários

Python é uma linguagem de programação conhecida por sua simplicidade e legibilidade, e uma de suas características mais poderosas é o suporte a estruturas de dados flexíveis e eficientes. Entre as estruturas de dados mais utilizadas em Python estão as tuplas, listas, dicionários e sets. Cada uma dessas estruturas tem suas próprias características, vantagens e casos de uso específicos. São estruturas de dados embutidas que permitem armazenar e manipular coleções de dados de forma eficiente. A seguir são apresentadas cada uma dessas estruturas, entendendo como elas funcionam, como são criadas e manipuladas, e em quais situações são mais adequadas.

Uma tupla é uma estrutura de dados em Python que permite armazenar vários elementos em uma única variável. Uma tupla é uma coleção ordenada e imutável

de elementos. Isso significa que, uma vez criada, uma tupla não pode ser alterada. Na sua sintaxe, as tuplas são definidas usando parênteses () e os elementos são separados por vírgulas.

```
minha _ tupla = (1, 2, 3, 4)
```

Algumas das características apresentadas nessa estrutura é a imutabilidade, pois após a criação, os elementos não podem ser adicionados, removidos ou alterados. Também a ordenação, onde os elementos mantêm a ordem em que foram inseridos. Por fim, a heterogeneidade, na qual pode conter elementos de diferentes tipos. Tuplas são ideais para armazenar dados que não devem ser modificados, como coordenadas geográficas, datas, ou configurações fixas.

O desempacotamento de tuplas permite atribuir os elementos de uma tupla a variáveis separadas. Nesse caso apresentado no exemplo a seguir, as variáveis “a”, “b” e “c” receberão os valores 1, 2 e 3, respectivamente. Isso pode ser útil quando você precisa trabalhar com tuplas de tamanho conhecido. Neste mesmo exemplo foi atribuído também somente o segundo valor da tupla a uma variável.

```
tupla = (13, 23, 33)
a, b, c = tupla
elemento2 = tupla[1]
print(f”a={a}, b={b}, c={c} e segundo elemento = {elemento2}”)
# Saída após a execução
a=13, b=23, c=33 e segundo elemento = 23
```

Uma interessante aplicação lidando com dados é a mescla das informações, é possível fazer a partir da função zip, conforme exemplo a seguir:

```
alunos = (‘Fulano’, ‘Ana’, ‘Vitor’)
G1 = (5.0, 9.1, 9.2)
G2 = (4.5, 8.9, 9.5)
situacao = (‘Substituição’, ‘Aprovado’, ‘aprovado’)
semestre = zip(alunos, G1, G2, situacao)
print(list(semestre))
# Saída após a execução
```

```
[('Fulano', 5.0, 4.5, 'Substituição'), ('Ana', 9.1, 8.9,
'Aprovado'), ('Vitor', 9.2, 9.5, 'aprovado')]
```

Uma lista é uma coleção ordenada e mutável de elementos. Diferente das tuplas, as listas podem ser modificadas após a criação. As listas são definidas usando colchetes [] e os elementos são separados por vírgulas.

```
minha _ lista = [1, 2, 3, 4]
```

Dentre as principais características é possível encontrar a mutabilidade onde os elementos podem ser adicionados, removidos ou alterados. A ordenação onde os elementos mantêm a ordem em que foram inseridos. A heterogeneidade, onde a lista pode conter elementos de diferentes tipos.

Listas são amplamente utilizadas para armazenar coleções de itens que podem mudar ao longo do tempo, como uma lista de tarefas, registros de dados, ou resultados de cálculos. É importante salientar que na disciplina de Estrutura de Dados, presente nos cursos da área de TI, há um estudo profundo de listas. As listas possuem uma variedade de métodos integrados, dentre os mais utilizados há o append() para adicionar elementos e o remove() para removê-los. A seguir, um exemplo da utilização dos mesmos.

```
minha _ lista = ["segunda", "terça","quarta"]

minha _ lista.append("quinta")

print(f"Print1 - {minha _ lista}")

minha _ lista.remove(minha _ lista[2]) #remoção pela posição

print(f"Print2 - {minha _ lista}")

minha _ lista.remove("terça") #remoção pelo valor

print(f"Print3 - {minha _ lista}")

# Saída após a execução

Print1 - ['segunda', 'terça', 'quarta', 'quinta']

Print2 - ['segunda', 'terça', 'quinta']

Print3 - ['segunda', 'quinta']
```

Outra estrutura importante em Python é o Dicionário. Um dicionário é uma coleção não ordenada de pares chave-valor. Cada elemento em um dicionário é acessado por meio de uma chave única. Na sua sintaxe os dicionários são definidos usando chaves {} e os pares chave-valor são separados por vírgulas. A chave e o valor são separados por dois pontos :.

```
Dicionário ={'chave':'items'}

#exemplo:

informacao = {'nome': 'Fulano', 'peso': 70, 'cidade': 'Itú'}
```

As principais características dessa estrutura estão no acesso, nos dicionários os valores são acessados por meio de chaves únicas. Os valores podem ser alterados, e novos pares chave-valor podem ser adicionados ou removidos, essa caraterística é conhecida como Mutabilidade. A partir do Python 3.7, a ordem de inserção é mantida. Dicionários são ideais para armazenar dados que precisam ser acessados rapidamente por uma chave única, como informações de usuários, configurações, ou mapeamentos de dados. A seguir um exemplo do uso dessa estrutura, além dos métodos integrados values(), keys e items, há uma inserção do item PIB no dicionário:

```
#exemplo:

informacao = {'Cidade':'Canoas', 'Habitantes':600000,'Infectados':100}

print(informacao.values())

print(informacao.keys())

print(informacao.items())

print(informacao["Cidade"])

informacao['PIB'] = 3000000.01

print(informacao)

# Saída após a execução

dict _ values(['Canoas', 600000, 100])

dict _ keys(['Cidade', 'Habitantes', 'Infectados'])

dict _ items([('Cidade', 'Canoas'), ('Habitantes', 600000), ('Infectados', 100)])

Canoas
```

```
{'Cidade': 'Canoas', 'Habitantes': 600000, 'Infectados': 100,
'PIB': 3000000.01}
```

Uma aplicação importante é criar listas de dicionários, a seguir um exemplo onde a lista é relacionada aos colchetes [] e os dicionários as chaves {}.

```
cidade1 = {'nome':'Canoas','Habitantes':700000}

cidade2 = {'nome':'Montenegro','Habitantes':60000}

covid = []

covid.append(cidade1)

covid.append(cidade2)

print(covid[0])

print(covid[1]['nome'])

# Saída após a execução

{'nome': 'Canoas', 'Habitantes': 700000}

Montenegro
```

Um set é uma coleção não ordenada de elementos únicos. Isso significa que não pode haver duplicatas em um set. Como sintaxe os sets são definidos usando chaves {} ou a função set(), e os elementos são separados por vírgulas. Dentre suas características podemos listar a Unicidade, Não Ordenado e a Mutabilidade. Unicidade porque cada elemento em um set é único. Não ordenados porque os elementos não têm uma ordem específica. Mutabilidade porque os elementos podem ser adicionados ou removidos, mas os próprios elementos devem ser imutáveis.

```
meu _ set = {1, 2, 3, 4}
```

Sets são úteis para operações que envolvem a eliminação de duplicatas, testes de pertinência, e operações de conjuntos como união, interseção e diferença.

Cada uma dessas estruturas de dados em Python tem suas próprias características e é adequada para diferentes tipos de problemas. Compreender quando e como usar tuplas, listas, dicionários e sets é fundamental para escrever código Python eficiente e legível. Ao dominar essas estruturas, você estará bem equipado para lidar com uma ampla variedade de desafios de programação.

4.4 Introdução a Manipulação de Arquivos

A manipulação de arquivos é uma parte essencial da programação, permitindo que se leia, escreva e gerencie dados em arquivos. Python oferece uma variedade de ferramentas e métodos para trabalhar com arquivos. Abaixo está uma introdução sobre como manipular arquivos em Python.

Para trabalhar com arquivos, é necessário abri-los usando a função open(). Essa função retorna uma referência ao arquivo, que pode ser usada para ler ou escrever dados. Sintaxe da função opan():

```
arquivo = open(nome _ do _ arquivo, modo)

#nome _ do _ arquivo: Caminho do arquivo (path) e o nome

#modo: como o arquivo será aberto (leitura, escrita, etc.)

#modos comuns:

     'r': Leitura (padrão). O arquivo deve existir.

     'w': Escrita. Se o arquivo existir, será sobrescrito.

                    Se não existir, será criado.

     'a': Adicionar. Escreve no final do arquivo sem apagar

                    o conteúdo existente.

     'b': Modo binário (usado com 'r', 'w' ou 'a' para trabalhar
com arquivos binários (não txt)).

     'x': Criação exclusiva. Falha se o arquivo já existir.

     '+': Atualização (leitura e escrita).

Exemplo:

arquivo = open("exemplo.txt", "r")

# essa linha abre exemplo.txt para leitura, esse arquivo deve
estar na mesma pasta do script, senão deve constar o path
```

Após abrir o arquivo, é possível ler o conteúdo usando métodos como `read()`, `readline()` ou `readlines()`. O método read() lê todo o conteúdo do arquivo como uma

string. O readline() lê uma linha do arquivo por vez, enquanto readlines() lê todas as linhas do arquivo e retorna uma lista de strings. A seguir são apresentados exemplos do uso das mesmas:

```
#exemplo read():

arquivo = open("exemplo.txt", "r")

conteudo = arquivo.read()

print(conteudo)

arquivo.close()


#exemplo readline():

arquivo = open("exemplo.txt", "r")

linha = arquivo.readline()

while linha:

   print(linha, end="")  # end="" evita quebras de linha extras

    linha = arquivo.readline()

arquivo.close()


#exemplo readlines():

arquivo = open("exemplo.txt", "r")

linhas = arquivo.readlines()

for linha in linhas:

    print(linha, end="")

arquivo.close()

#Se for utilizado o mesmo arquivo exemplo.txt para executar os 3 exemplos, as saídas serão iguais e mostrarão o conteúdo do arquivo.
```

Para escrever em um arquivo, é necessário utilizar o modo ‘w’ (sobrescreve) ou o modo ‘a’ (adiciona). O método write() escreve uma string no arquivo. Enquanto o writelines() escreve uma lista de strings no arquivo.

```
#Exemplo do uso de write:

arquivo = open("exemplo.txt", "w")

arquivo.write("Olá, mundo!\n")

arquivo.write("Python é incrível.\n")

arquivo.close()


#Exemplo do uso de writelines():

linhas = ["Linha 1\n", "Linha 2\n", "Linha 3\n"]

arquivo = open("exemplo.txt", "w")

arquivo.writelines(linhas)

arquivo.close()
```

Conforme apareceu nos exemplos anteriores, sobre manipulação de arquivos, quando o arquivo é aberto (open()) é importante que seja fechado (close()). As instruções de manipulação de arquivo são executadas com os dados em memória, o salvamento das mesmas só se dá quando a função de fechar é executada, ou alguma função para salvar sem fechar. Além disso, fechar o arquivo após o uso libera recursos do sistema, como memória, por exemplo. Isso pode ser feito com o método close(). Exemplo:

```
arquivo = open("exemplo.txt", "r")

conteudo = arquivo.read()

arquivo.close()
```

Uma alternativa para tentar proteger os dados caso haja algum problema é o uso de with, alternativa recomendada. O bloco with fecha o arquivo automaticamente, mesmo que ocorra uma exceção. Exemplo:

```
with open("exemplo.txt", "r") as arquivo:

    conteudo = arquivo.read()

# O arquivo é fechado automaticamente aqui
```

Nem sempre os arquivos trabalhados são arquivos de texto, organizados como uma sequência de caracteres conhecida. Muitos dos arquivos que é necessário manipular têm organização de dados própria, mesmo que sejam padrões, como por exemplo imagens, arquivos compactados, arquivos do pacote office (isso mesmo, um .docx não é um arquivo texto), arquivos executáveis, por exemplo. São chamados de arquivos binários. Por ter um formato padrão de organização de dados em caracteres os arquivos de texto podem ser abertos em qualquer editor de texto, mas se um arquivo binário for aberto em um editor qualquer, como o bloco de notas ou o gedit, o conteúdo exibido será uma sequência sem sentido de caracteres. Esse assunto não será aprofundado aqui, mas é possível trabalhar com arquivos binários em Python. Para manipular arquivos binários é necessário usar o modo 'b' como argumento para open(). Exemplo de leitura e de escrita de arquivo binário:

```
#leitura:

with open("imagem.jpg", "rb") as arquivo:

     dados = arquivo.read()


#escrita:

with open("copia.jpg", "wb") as arquivo:

     arquivo.write(dados)
```

Além de arquivos, você pode manipular diretórios usando o módulo `os` ou `pathlib`.

```
#Listando arquivos em um Diretório:

import os


arquivos = os.listdir("caminho/do/diretorio")

for arquivo in arquivos:

     print(arquivo)


#Verificando se um Arquivo ou Diretório Existe

import os
```

```
if os.path.exists("caminho/do/arquivo.txt"):
    print("O arquivo existe!")

#Criando e Removendo Diretórios

import os

os.mkdir("novo _ diretorio")  # Cria um diretório
os.rmdir("novo _ diretorio")  # Remove um diretório vazio
```

A manipulação de arquivos em Python é simples e poderosa. Com as funções e métodos adequados, é possível ler, escrever e gerenciar arquivos de texto e binários de forma eficiente. Além disso, o uso de boas práticas, como o bloco `with` e o tratamento de exceções, garante que seu código seja seguro e robusto. Para projetos mais complexos, explore a biblioteca `pathlib` (para manipulação de caminhos).

4.5 Projetos com Mais de um Arquivo Fonte

Um módulo é um arquivo Python (‘.py’) que contém funções, classes e variáveis que podem ser reutilizados em outros programas. Neste momento do aprendizado é indicado se concentrar no conceito de funções, trabalhadas nessa unidade. Para criar um módulo basta definir funções, classes ou variáveis em um arquivo `.py` e importar o módulo em outros scripts usando a instrução `import`. A seguir um exemplo.

```
# arquivo: meu _ modulo.py
def saudacao(nome):
     return f”Olá, {nome}!”


def soma(a, b):
     return a + b



# arquivo: main.py
import meu _ modulo


print(meu _ modulo.saudacao(“João”))
print(meu _ modulo.soma(2, 3))


#Saída após a execução
# Olá, João!
# 5
```

Além dos módulos, também é possível desenvolver Pacotes, que são coleções de módulos organizados em diretórios. Eles permitem uma estrutura hierárquica para organizar projetos maiores. Para criar um pacote crie um diretório com um arquivo ‘__init__.py’ (pode estar vazio ou conter inicializações). A seguir, adicione módulos dentro desse diretório. Exemplo:

```
#diretório meu _ pacote/
#arquivo _ _ init _ _ .py
```

```
modulo1.py

modulo2.py



# arquivo: main.py

from meu _ pacote import modulo1, modulo2


modulo1.funcao1()

modulo2.funcao2()
```

Dentro deste contexto, uma demanda importante é o uso de Bibliotecas Externas, pois Python possui um ecossistema rico de bibliotecas externas que podem ser instaladas via 'pip'. Essas bibliotecas são modularizadas e permitem adicionar funcionalidades complexas aos projetos sem reinventar a roda. Exemplo:

```
pip install numpy


import numpy as np

my _ array = np.array([1, 2, 3, 4, 5])

print(my _ array)

#Saída após execução

[1 2 3 4 5]
```

A organização de projetos com estrutura de pastas é importante para projetos maiores, é comum organizar o código em pastas separadas para módulos, testes, recursos, etc. Exemplo de estrutura:

```
meu _ projeto/

     main.py

     meu _ pacote/

          _ _ init _ _ .py
```

```
            modulo1.py
            modulo2.py
tests/
            test _ modulo1.py
            test _ modulo2.py
```

Também é possível a criação de bibliotecas próprias, caso existam funções que são úteis em vários projetos, pode empacotá-las como uma biblioteca Python e instalá-las via 'pip'. Passos:

1. Crie uma estrutura de diretórios para sua biblioteca.
2. Adicione um arquivo `setup.py` para configurar o pacote.
3. Publique no PyPI (Python Package Index) ou instale localmente.

```
#Exemplo de `setup.py':
from setuptools import setup, find _ packages


setup(
    name="minha _ biblioteca",
    version="0.1",
    packages=find _ packages(),
)
```

Para criar documentação de módulos e funções é possível utilizar docstrings (são strings inseridas no código Python com o intuito de fornecer uma explicação sobre o funcionamento deste). Isso facilita a compreensão e o uso do código por outros desenvolvedores (ou mesmo para o próprio desenvolvedor no futuro).

```
def calcular _ area _ retangulo(largura, altura):
    """
    Calcula a área de um retângulo.

    :param largura: Largura do retângulo.
    :param altura: Altura do retângulo.
    :return: Área do retângulo.
    """
    return largura * altura
```

A programação modular em Python vai muito além das funções embutidas. Ao utilizar módulos, pacotes, classes, bibliotecas externas, testes e boas práticas de organização, o programador pode criar sistemas robustos, escaláveis e de fácil manutenção. A chave é dividir o código em partes menores e reutilizáveis, seguindo os princípios de modularidade e encapsulamento.

Síntese da Unidade

Aqui está um resumo dos principais temas abordados nas 5 seções desta Unidade, sobre condicionais, laços e matrizes.

1. Questões fundamentais sobre programação modular
    - Introdução sobre a importância da programação Modular.
    - Apresenta as principais funções embutidas do Python (Built-in).
2. Formas de uma função
    - Apresenta a sintaxe para criação de funções.
    - Apresenta exemplos de criação de funções.
3. Tuplas, Listas e Dicionários
    - Apresenta estas importantes estruturas, além disso, os Sets.
    - Apresenta a sintaxe e exemplos destas estruturas.
4. Introdução a manipulação de arquivos.
    - Introduz o trabalho com arquivos.
    - Apresenta exemplos com arquivos texto e binários
5. Projetos com mais de um arquivo fonte
    - Introduz o trabalho com módulos e bibliotecas de arquivos.
    - Apresenta exemplos deste trabalho.

Esse resumo abrange os temas principais e fornece uma visão geral do conteúdo abordado na unidade

Bibliografia

BANIN, S. L. **Python 3 - Conceitos e Aplicações - Uma abordagem didática**. Rio de Janeiro: Érica, 2018. E-book. p.Capa. ISBN 9788536530253. Disponível em: minhabiblioteca.com.br/reader/books/9788536530253/. Acesso em: 15 out 2024.

DEEPSEEK. Disponível em: https://deepseek.com/. Acesso em: 25 fev. 2025.

LAMBERT, Kenneth A. **Fundamentos de Python:** primeiros programas. São Paulo: Cengage Learning Brasil, 2022. E-book. p.i. ISBN 9786555584301. Disponível em: minhabiblioteca.com.br/reader/books/9786555584301/. Acesso em: 15 out 2024.

NUMPY. Disponível em: https://numpy.org/. Acesso em: 25 fev. 2025.

SEVERANCE, C. R. **Python para Todos.** CreateSpace Independent Publishing Platform. 2016. Disponível em: https://do1.dr-chuck.com/pythonlearn/PT_br/pythonlearn.pdf. Acesso em: 15 out 2024.

RAMALHO, L.. A**prenda a Programar.** Creative Commons. Disponível em: https://wiki.python.org.br/AprendaProgramar ou em . Acesso em: 14 fev 2025.

POE. Disponível em: https://poe.com/. Acesso em: 25 fev. 2025.

PYTHON. Documentação. Disponível em: https://www.python.org/. Acesso em: 25 fev. 2025.

WAINER, J. **Expressões Relacionais, Lógicas, Comandos Condicionais e Funções.** Disponível em: https://www.ic.unicamp.br/~wainer/cursos/1s2024/102/aula03.html. Acesso em: 25 fev. 2025.

CONTRIBUA COM A QUALIDADE DO SEU CURSO

Se você encontrar algum problema neste material, entre em contato pelo email **eadproducao@unilasalle.edu.br.** Descreva o que você encontrou e indique a página.

Lembre-se: a boa educação se faz com a contribuição de todos!

UNIVERSIDADE LaSalle

Viver é **evoluir**.

Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500
eadproducao@unilasalle.edu.br